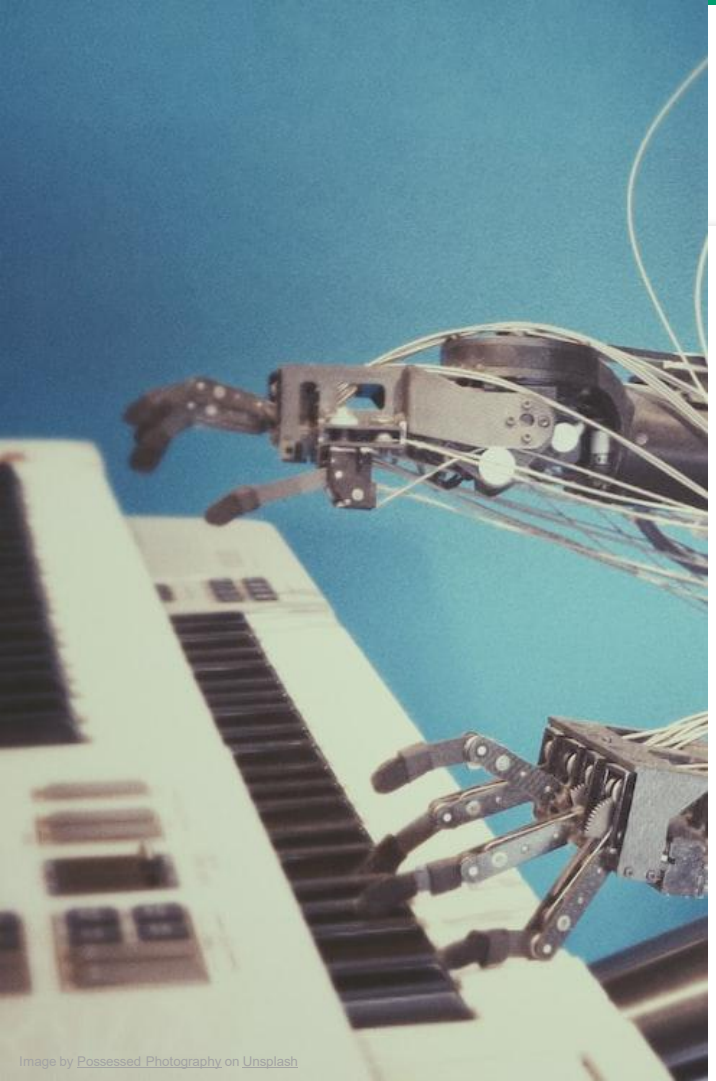


Applied Artificial Intelligence
04 - AI Lifecycle: Deployment

Univ.-Prof. Dr-Ing. habil. Niklas Kühl
www.niklas.xyz

University of Bayreuth

TUM School of Management

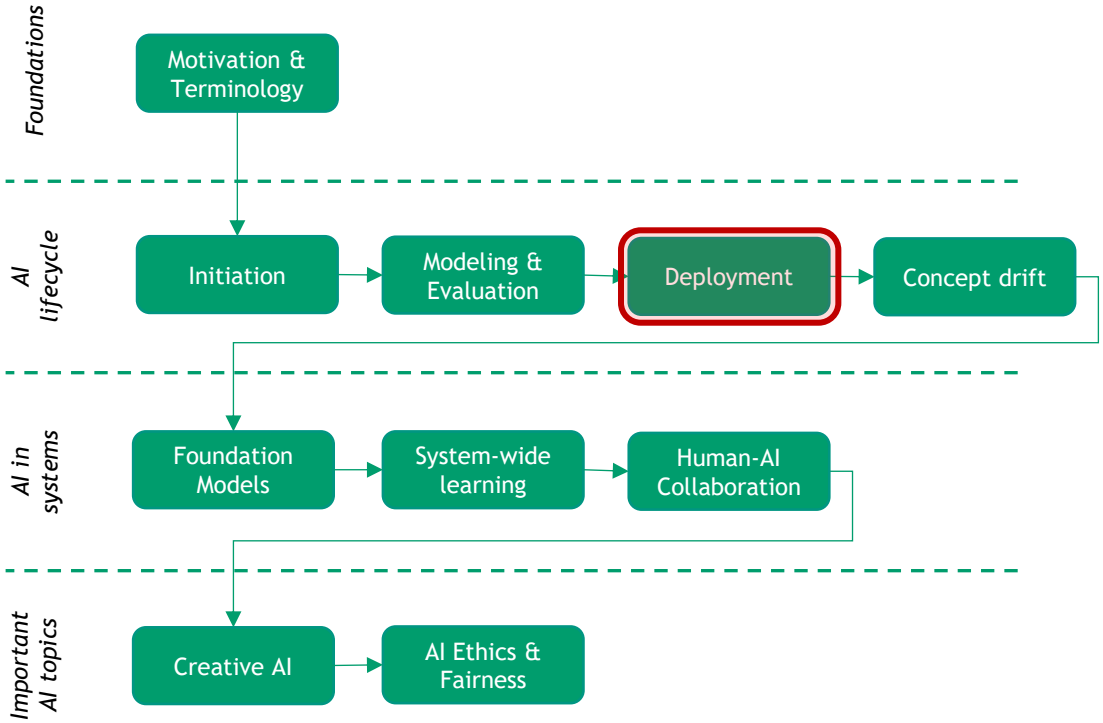
Applied Artificial Intelligence © 2024 by Niklas Kühl is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International.

Repetition

Capability of data & algorithm	There are multiple concepts for supervised machine learning. The performance of each algorithm is determined by the **data and** the **algorithms' capabilities** to capture patterns in that very data.
Model optimization	**Models** can be **optimized** by tuning the **parameters** of the underlying algorithm.
Performance selection	The **selection** of the performance metric sets the **aim** for the model.
Interplay of components	It is important to understand the **interplay** of **problem statement**, **data (distribution)**, **algorithm** and **performance metric**.

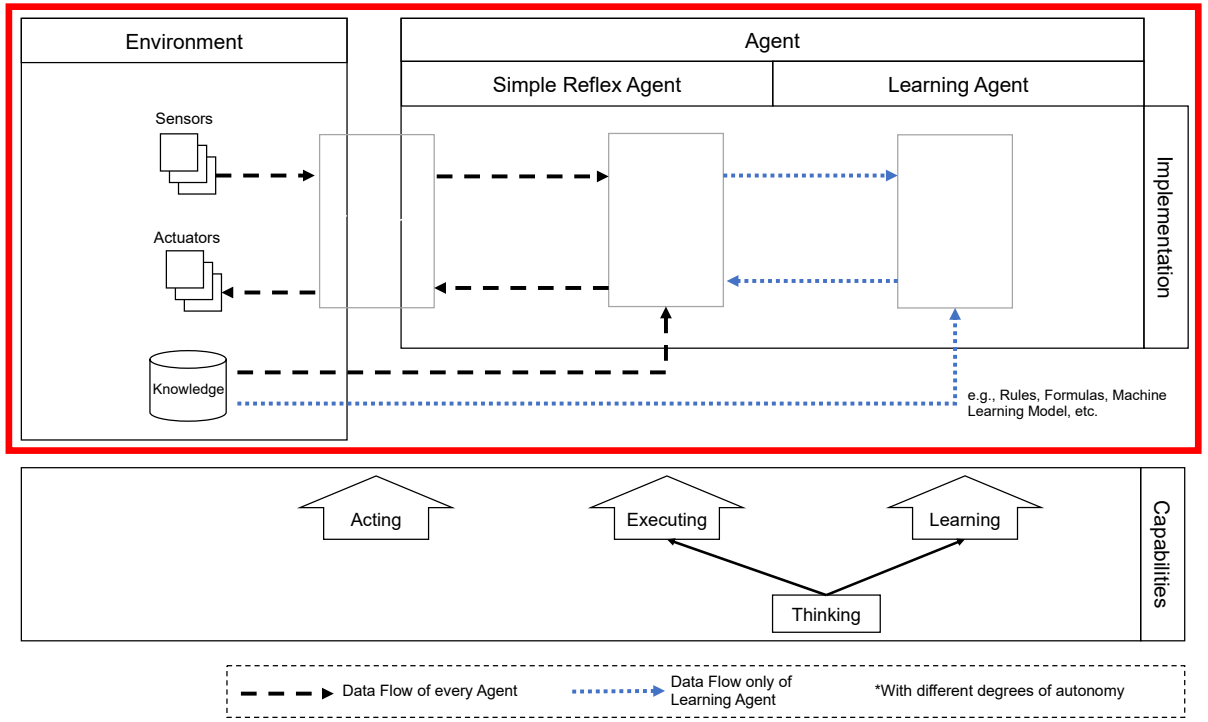
Prof. Dr. Niklas Kühl

Organizational
The story of the lecture.



Prof. Dr. Niklas Kühl

Positioning of the “deployment” step within an intelligent agent’s architecture.

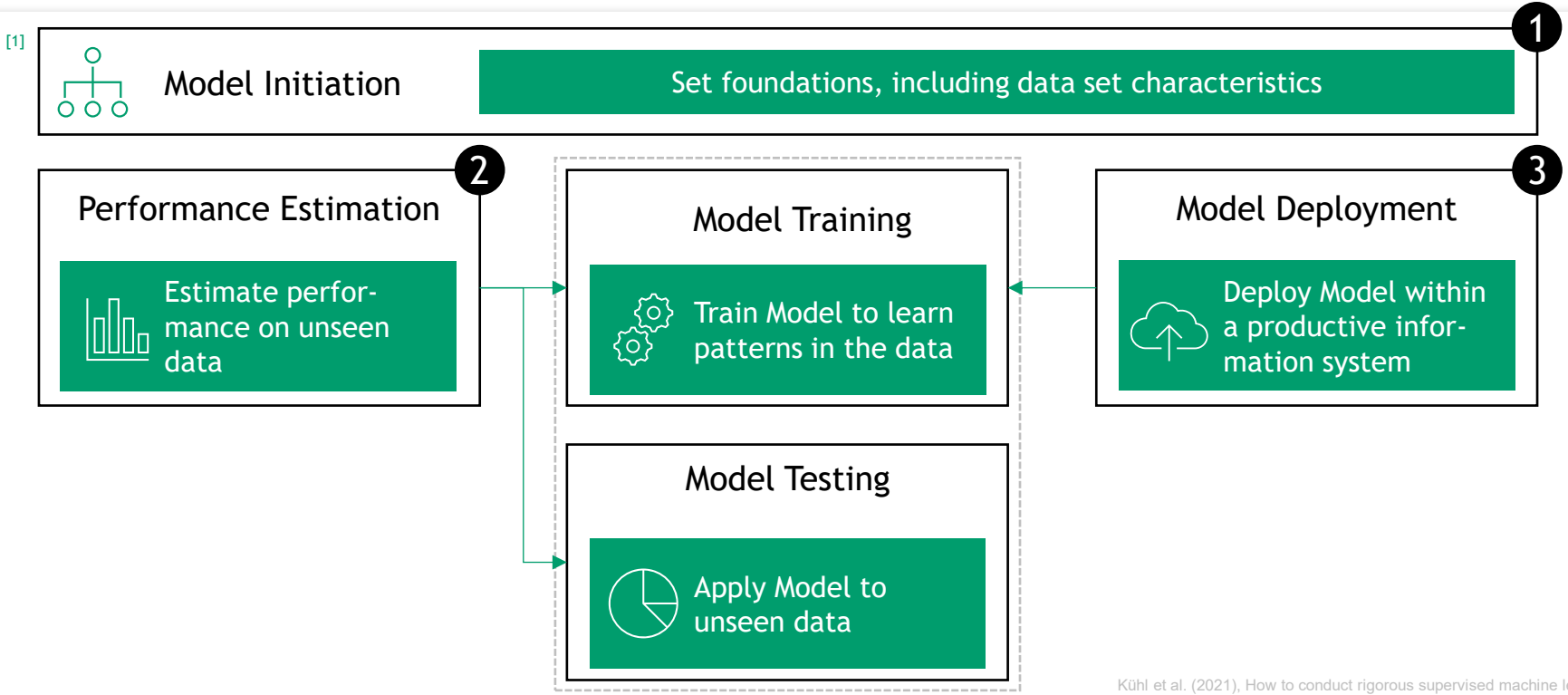


[1]

Kühl et al. (2022), Artificial intelligence and machine learning [1]

Prof. Dr. Niklas Kühl

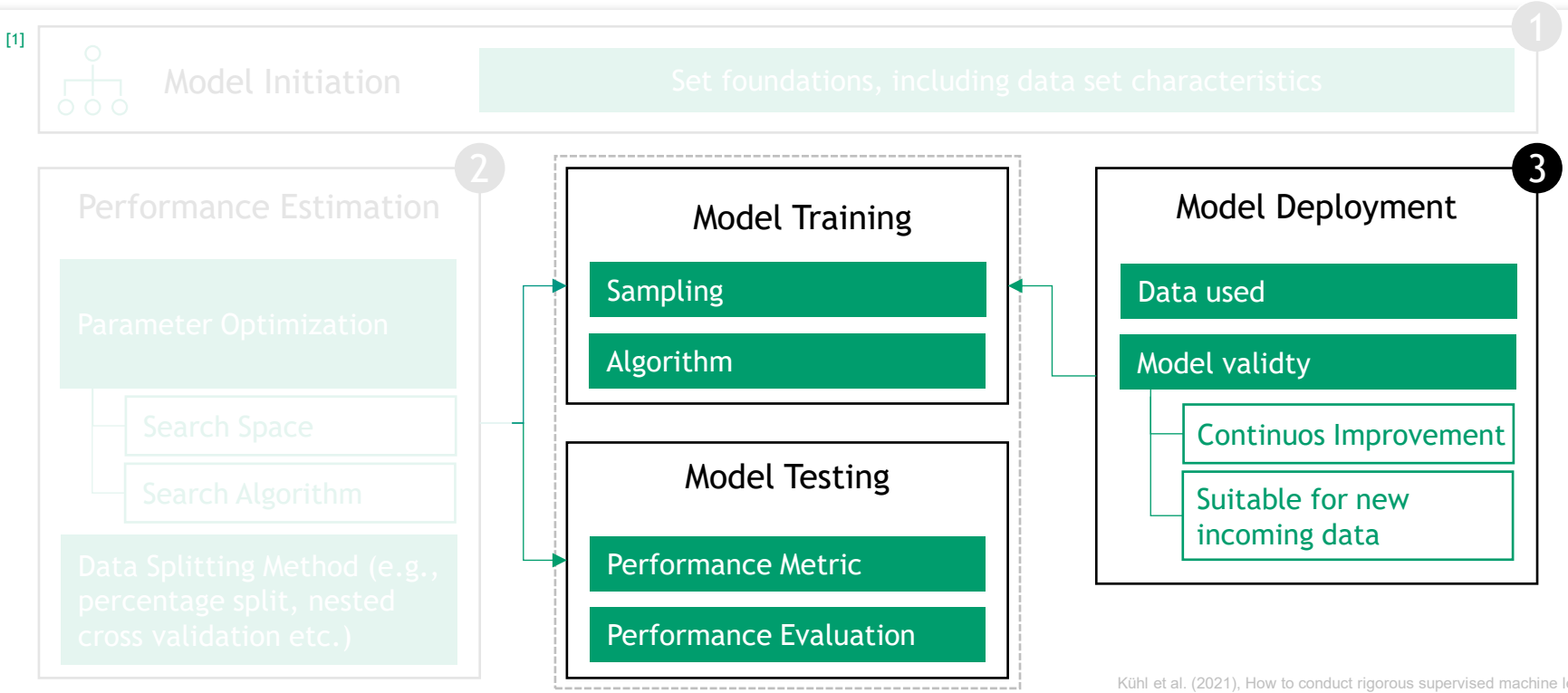
The AI Lifecycle: Required steps for supervised machine learning in the deployment step.



Kühl et al. (2021), How to conduct rigorous supervised machine learning in information systems research: the supervised machine learning report card [1]

Prof. Dr. Niklas Kühl

The AI Lifecycle: Required steps for supervised machine learning in the deployment step.



Kühl et al. (2021), How to conduct rigorous supervised machine learning in information systems research: the supervised machine learning report card [1]

Prof. Dr. Niklas Kühl

Objectives
What are the learning goals of this lecture?

EXPLORE	UNDERSTAND	INTENSIFY	APPLY
Locate deployment in the AI system lifecycle	Explain how capability, authority, architecture and runtime interact	Evaluate observability, governance and release controls	Design a controlled path from prediction to physical action.






Prof. Dr. Niklas Kühl

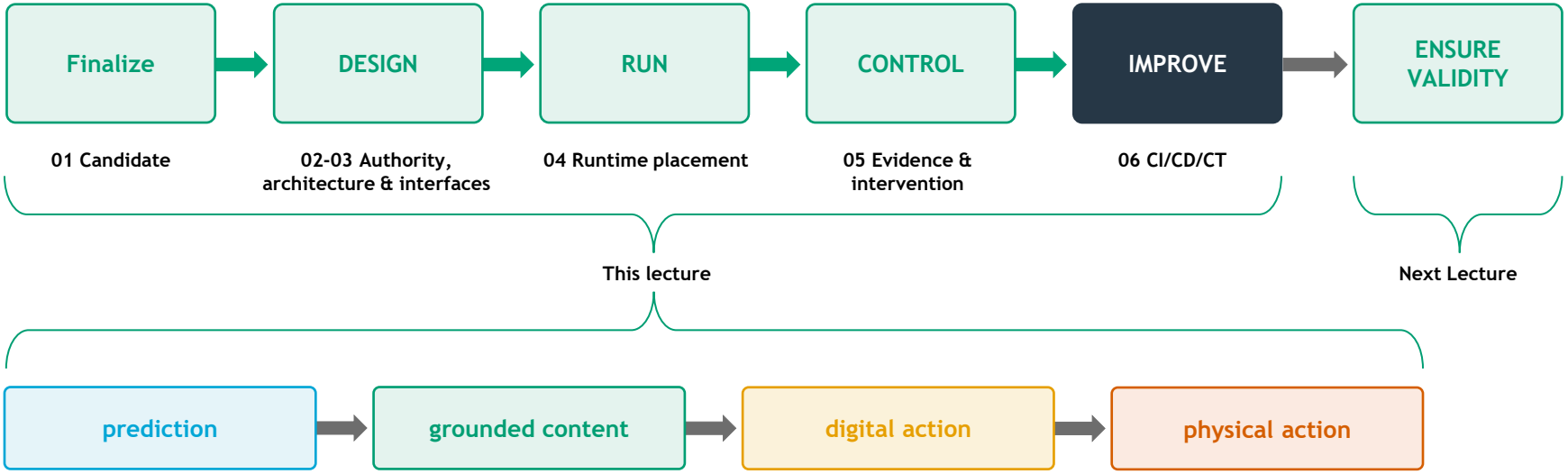


1. From Model to Deployment
2. Levels of Autonomy in AI Deployment
3. AI System Architectures & Interfaces
4. Infrastructure & Runtime Options
5. Monitoring & Runtime Governance
6. MLOps: Continuous Integration, Delivery & Training

Process
This lecture focuses on deployment of the final model.



AI deployment is a lifecycle of system processes and continuous risk management. [1][2][3]



NIST (2023), Artificial Intelligence Risk Management Framework [1]
ISO (2023), ISO/IEC 5338:2023: Information technology—Artificial intelligence—AI system life cycle processes [2]
ISO (2022), ISO/IEC 23053:2022: Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML) [3]

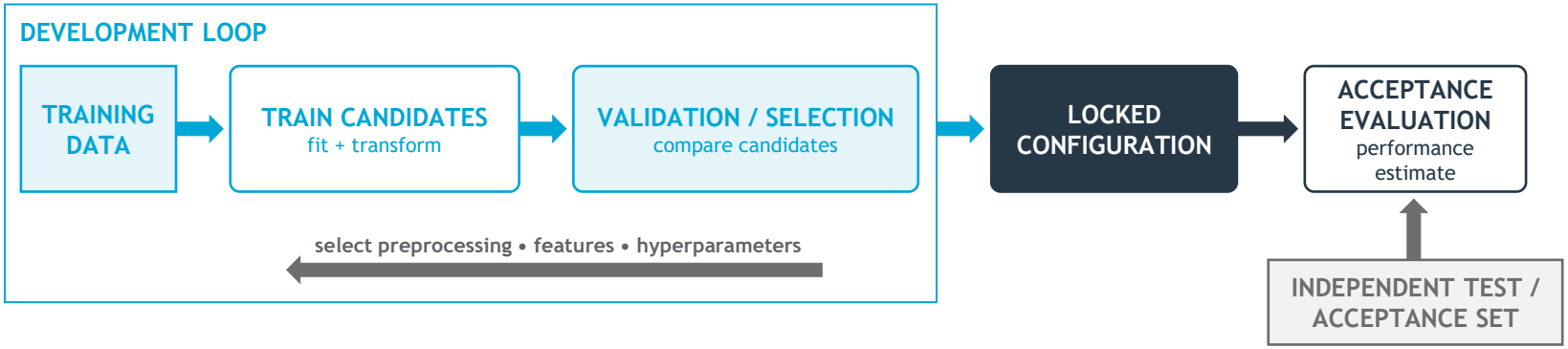
Prof. Dr. Niklas Kühl



1. From Model to Deployment
2. Levels of Autonomy in AI Deployment
3. AI System Architectures & Interfaces
4. Infrastructure & Runtime Options
5. Monitoring & Runtime Governance
6. MLOps: Continuous Integration, Delivery & Training

From Model to Deployment
Model selection and performance estimation must remain separated.

Selection occurs inside the development loop;
Acceptance evidence stays outside it. [1]



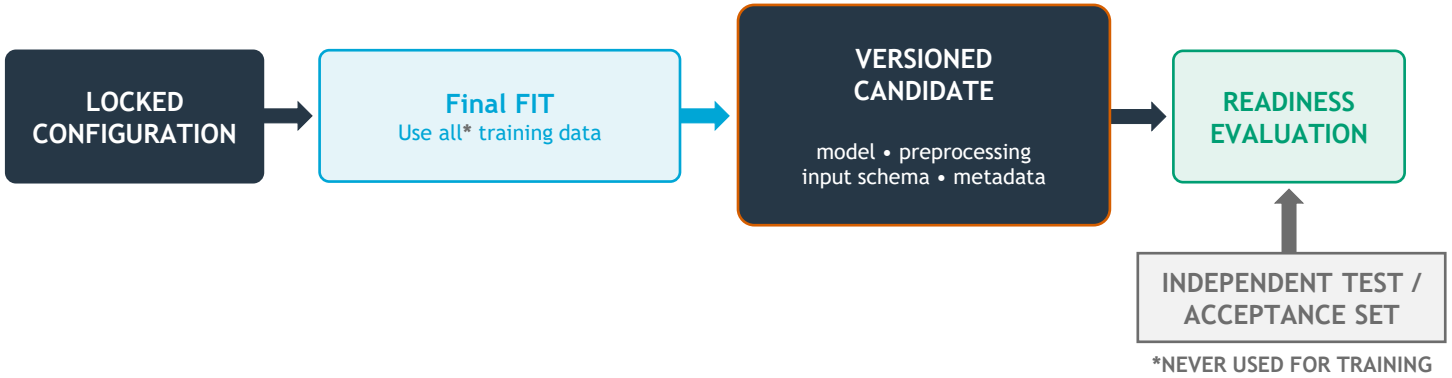
Repeated selection on the same evaluation data can **overfit** the selection **procedure** itself. [1]

Cawley et al. (2010), On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation [1]

From Model to Deployment

Final training creates a versioned candidate—not a production decision.

Reproducible artifacts and validation evidence precede production readiness. [1][2][3]

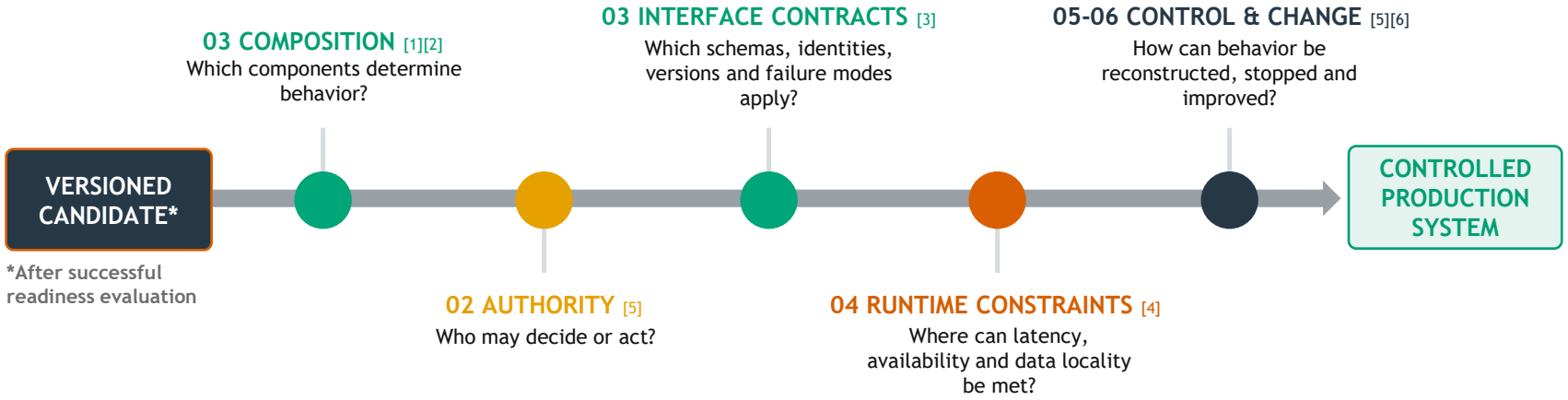


Baylor et al. (2017), TFX: A TensorFlow-Based Production-Scale Machine Learning Platform [1]
Breck et al. (2017), The ML Test Score: A rubric for ML Production Readiness and Technical Debt Reduction [2]
ISO (2023), ISO/IEC 5338:2023: Information technology—Artificial intelligence—AI system life cycle processes [3]

Prof. Dr. Niklas Kühl

From Model to Deployment
Deployment challenges arise at the boundaries around the model.

A capable model becomes deployable only when its surrounding system is composable, constrained and controllable.



An accurate **model can** still **be undeployable.** [1][2]

Sculley et al. (2015), Hidden Technical Debt in Machine Learning Systems [1]
ISO (2022), ISO/IEC 23053:2022: Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML) [2]
OpenAPI (2025), OpenAPI Specification 3.2.0. [3]
Shi et al. (2016), Edge Computing: Vision and Challenges [4]
NIST (2023), Artificial Intelligence Risk Management Framework [5]
Kreuzberger et al. (2023), Machine Learning Operations (MLOps): Overview, Definition and Architecture [6]

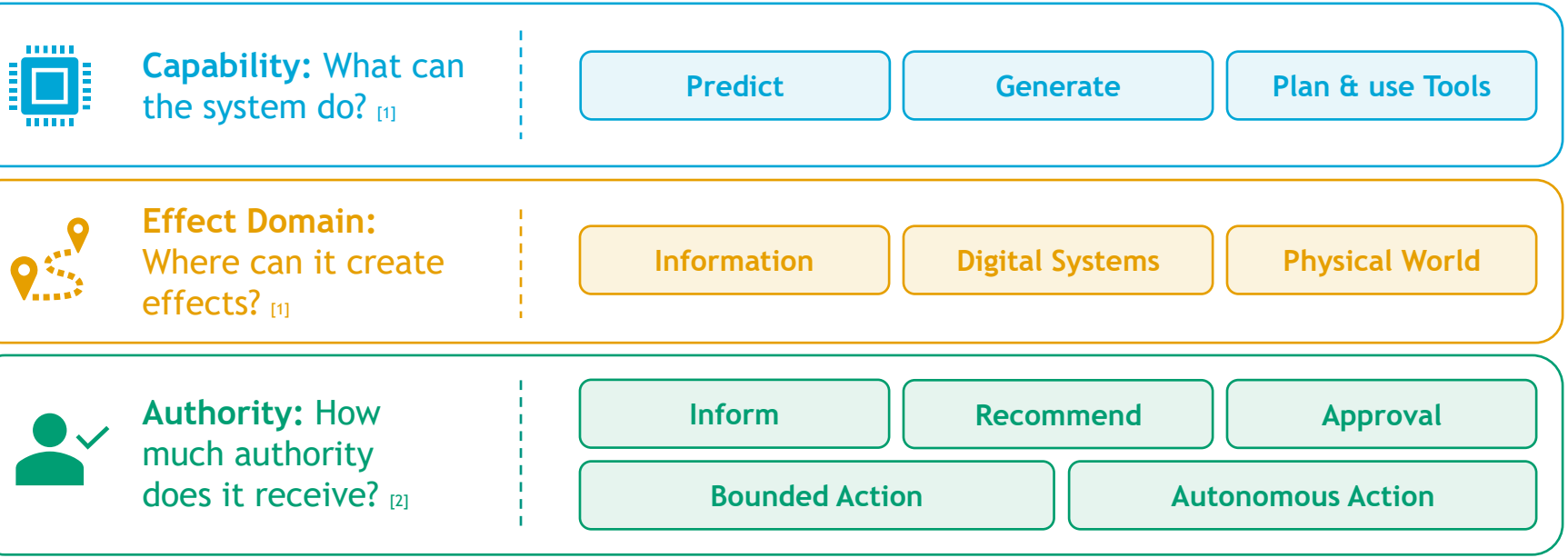
Prof. Dr. Niklas Kühl



https://images.unsplash.com/photo-1628258334105-2a0b3d6efee1?auto=format&fit=crop&q=80&w=3615&ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D

1. From Model to Deployment
2. Levels of Autonomy in AI Deployment
3. AI System Architectures & Interfaces
4. Infrastructure & Runtime Options
5. Monitoring & Runtime Governance
6. MLOps: Continuous Integration, Delivery & Training

Levels of Autonomy in AI deployment
Three questions for deployment design.



Capability, Effect Domain and **Authority** are separate dimensions. [1][2]

OECD (2022), Framework for the Classification of AI Systems [1]
Parasuraman et al. (2000), A Model for Types and Levels of Human Interaction with Automation [2]

Prof. Dr. Niklas Kühl

Levels of Autonomy in AI Deployment

Automation varies by function—and by degree.



- **Information acquisition:** E.g., machine learning model outputting prediction results
- **Information analysis:** E.g., machine learning model outputting recommendations

---

- **Decision and Action selection:** E.g., machine learning model directly taking actions in some cases
- **Action implementation:** E.g., machine learning model always taking actions

LOW
Human performs; AI assists

HIGH
AI performs; human supervises

Each function can be automated at a different level. [1]

Parasuraman et al. (2000), A Model for Types and Levels of Human Interaction with Automation [1]

Prof. Dr. Niklas Kühl

Levels of Autonomy in AI Deployment
Four archetypes connect capability to effect.

Predictive AI [1]
Core Output:
Score, class or forecast
Effect Domain:
Information

Generative AI [2]
Core Output:
Open-ended content
Effect Domain:
Information

Agentic AI [3]
Core Output:
Planned tool actions
Effect Domain:
Digital Systems

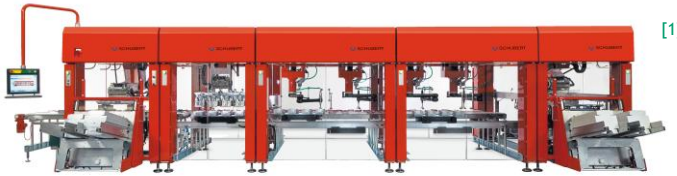
Physical AI [4]
Output:
Physical state change
Effect Domain:
Physical World

Archetypes are not mutually exclusive—they often overlap and combine!

OECD (2022), Classification of AI Systems [1]
Feuerriegel et al. (2024), Generative AI [2]
Allmendinger et al. (2026), Multi-agent AI [3]
ISO (2025), ISO 10218-1: Robotics—Safety requirements for industrial robots [4]

Prof. Dr. Niklas Kühl

Levels of Autonomy in AI Deployment
An example from practice.


[1]

Case A: No AI
10% of products faulty

Exemplary outcomes		
Output	Shipped to customer	Claimed by customer
100	100	-10

https://commons.wikimedia.org/wiki/File:TLM-Verpackungsmaschine_neueste_Generation.jpg [1]

Prof. Dr. Niklas Kühl

Levels of Autonomy in AI Deployment
Example from Schwarz/Kaufland.



Fig. 7 Expiry Date, lot number and Barcode defect



Fig. 8 Expiry Date and lot number wrong

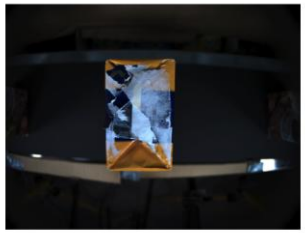
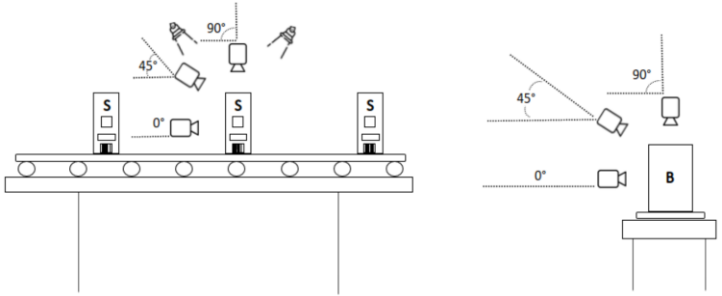


Fig. 9 Top Sticker missing [1]



01 Observe
Classify package defects [1]
- (Capability) **Predict**
- (Effect Domain) **Information**
- (Authority) **Inform**

02 Recommend
Flag package for review
- (Capability) **Predict**
- (Effect Domain) **Information**
- (Authority) **Recommend**

03 Act
Trigger a rejector
- (Capability) **Predict**
- (Effect Domain) **Physical World**
- (Authority) **Bounded Action**

The model can stay the same while effects and authority change. [2]

Xiong et al. (2023), Designing a computer-vision-based artifact for automated quality control: A case study in the food industry [1]
Parasuraman et al. (2000), A Model for Types and Levels of Human Interaction with Automation [2]

Prof. Dr. Niklas Kühl

Levels of Autonomy in AI Deployment

Greater authority creates stronger control obligations.

Predictive AI [1]

- Calibration & drift
- Data lineage
- Model versioning

Generative AI [2]

- Grounding
- Output validation
- Content provenance

Agentic AI [3][4]

- Least privilege
- Approval & stop
- Audit trail

Physical AI [5]

- Safety envelope
- Real-time guard
- Fallback state

Control requirements grow with potential impact and delegated authority. [4][5]

NIST (2023), Artificial Intelligence Risk Management Framework [1]
NIST (2024), Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile [2]
Allmendinger et al. (2026), Multi-agent AI [3]
ISO (2025), ISO 10218-1: Robotics—Safety requirements for industrial robots [4]
Parasuraman et al. (2000), A Model for Types and Levels of Human Interaction with Automation [5]

Prof. Dr. Niklas Kühl



1 From Model to Deployment

2 Levels of Autonomy in AI Deployment

3 AI System Architectures & Interfaces

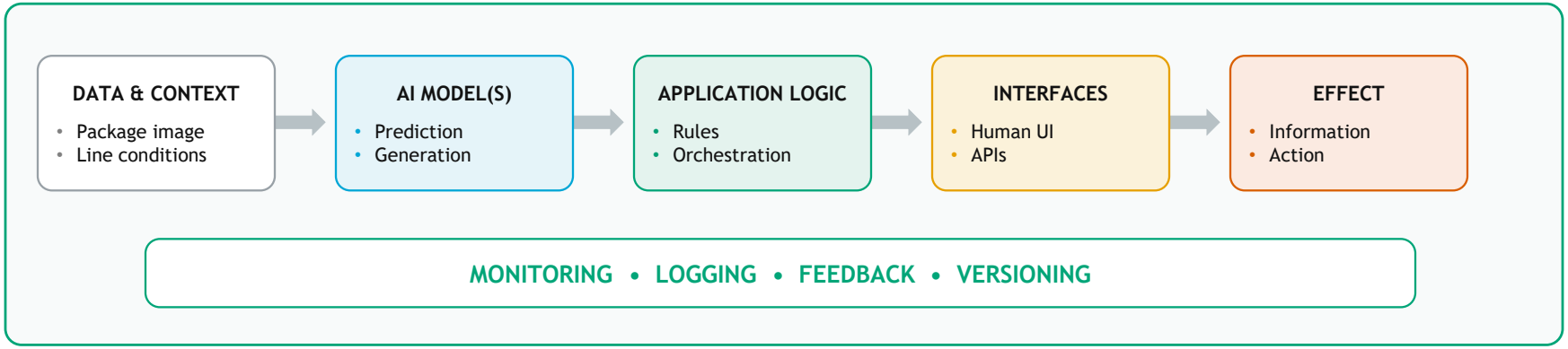
4 Infrastructure & Runtime Options

5 Monitoring & Runtime Governance

6 MLOps: Continuous Integration, Delivery & Training

AI System Architectures & Interfaces
A deployed AI application is a system, not a model.

In production, data, models, rules and interfaces form one operational unit. [1][2]



Deployment concerns the **end-to-end system**—not only model serving. [3]

Chaudhry et al. (2025), Towards Resource-Efficient Compound AI Systems [1]
Sculley et al. (2015), Hidden Technical Debt in Machine Learning Systems [2]
ISO (2022), ISO/IEC 23053:2022: Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML) [3]

Prof. Dr. Niklas Kühl

AI System Architectures & Interfaces
Predictive AI becomes a versioned decision service.

The model provides information;
Policy and human roles determine what happens next. [1]

Schwarz/Kaufland example: [3]



A **production contract** must identify **inputs**, **outputs** and the deployed **model version.** [2][3]

INPUT SCHEMA • PREDICTION + SCORE • MODEL VERSION • TRACE ID

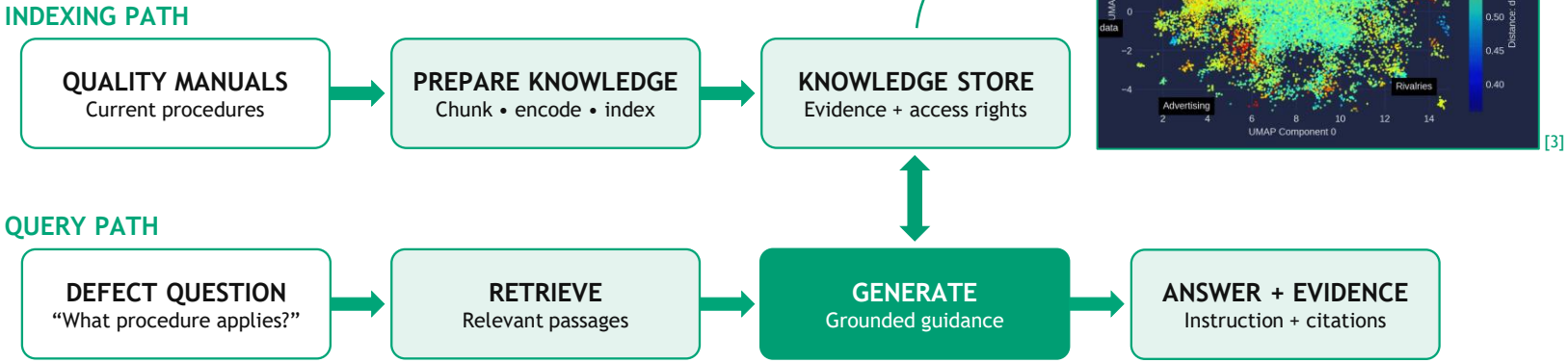
Parasuraman et al. (2000), A Model for Types and Levels of Human Interaction with Automation [1]
Baylor et al. (2017), TFX: A TensorFlow-Based Production-Scale Machine Learning Platform [2]
Xiong et al. (2023), Designing a computer-vision-based artifact for automated quality control: A case study in the food industry [3]

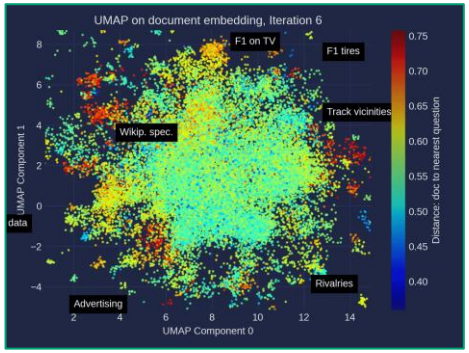
AI System Architectures & Interfaces

RAG adds a knowledge runtime beside the model.

RAG ≙ Retrieval-Augmented Generation

Retrieval combines parametric generation with an explicit, updateable knowledge source. [1]





[3]

Document freshness, access rights and **source traceability** become **deployment concerns.** [2]

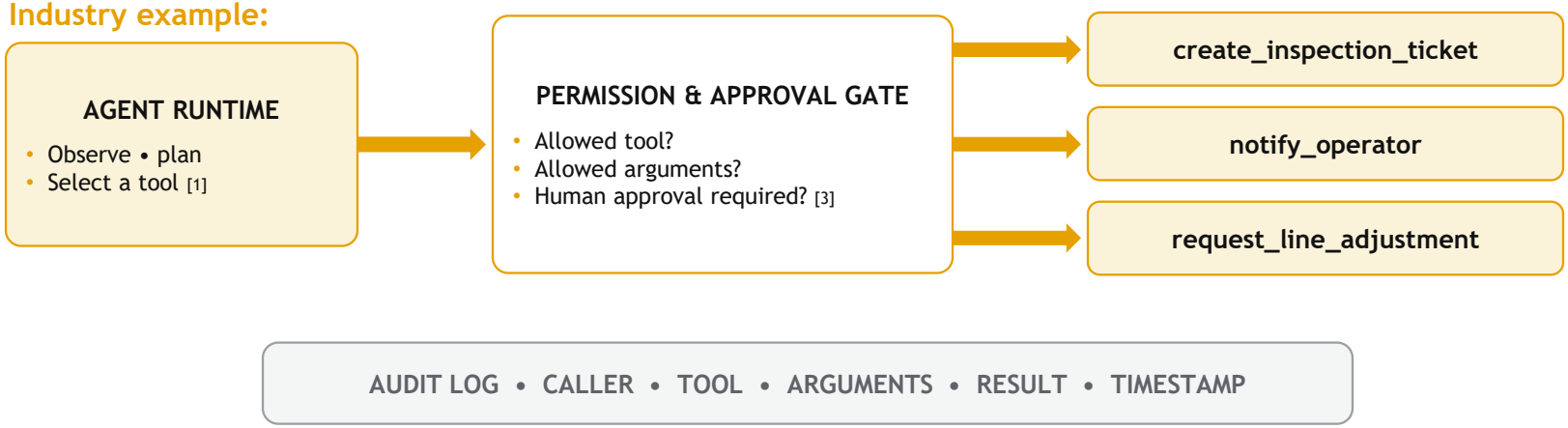
Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [1]
NIST (2024), Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile [2]
Stoll (2024), Visualize your RAG Data—Evaluate your Retrieval-Augmented Generation System with Ragas [3]

Prof. Dr. Niklas Kühl

AI System Architectures & Interfaces
Tool interfaces turn recommendations into bounded digital actions.

An agent can select and call tools through APIs, but execution needs explicit authority. [1][3]

Industry example:



A **tool call** is a transaction with **consequences**—not only model output. [2][3]

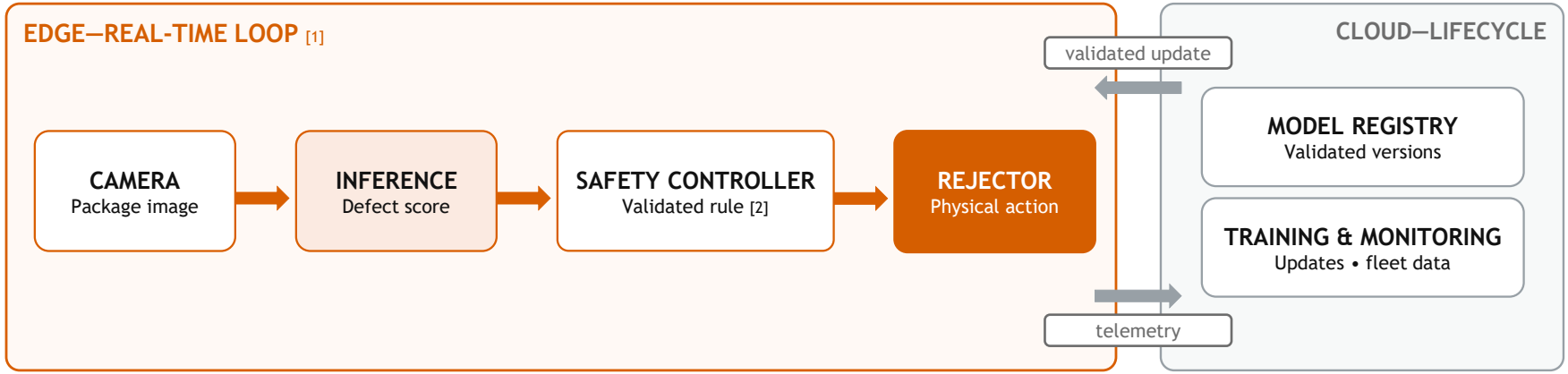
Schick et al. (2023), Toolformer: Language Models Can Teach Themselves to Use Tools [1]
Allmendinger et al. (2026), Multi-agent AI [2]
Parasuraman et al. (2000), A Model for Types and Levels of Human Interaction with Automation [3]

Prof. Dr. Niklas Kühl

AI System Architectures & Interfaces
Edge deployment closes the physical loop locally.

The safety-critical path stays close to the machine;
Lifecycle services can remain central. [1][2]

Schwarz/Kaufland example: [3]



Safety-related **action** remains governed by the **local safety controller**. [2][3]

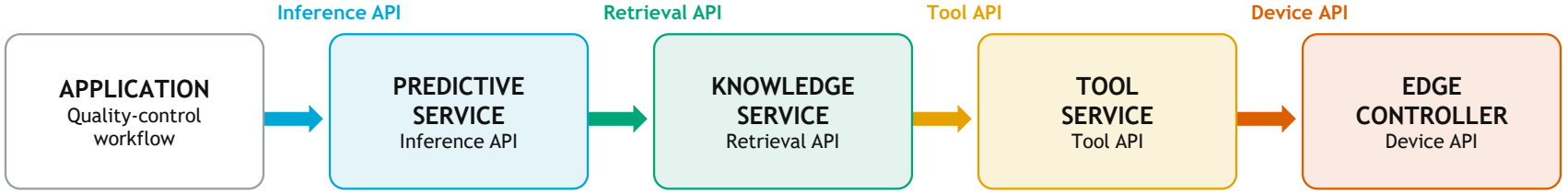
Shi et al. (2016), Edge Computing: Vision and Challenges [1]
ISO (2025), ISO 10218-1: Robotics—Safety requirements for industrial robots [2]
Xiong et al. (2023), Designing a computer-vision-based artifact for automated quality control: A case study in the food industry [3]

Prof. Dr. Niklas Kühl

AI System Architectures & Interfaces
APIs make heterogeneous AI components composable.

API ≙ Application Programming Interface

An API describes a component's capabilities without exposing its implementation. [1]



SCHEMA & SEMANTICS • IDENTITY & PERMISSIONS • ERRORS & TIMEOUTS • VERSION & TRACEABILITY [1][2]

Stable interfaces reduce coupling between **models**, **data** and **application logic.** [2][3]

OpenAPI (2025), OpenAPI Specification 3.2.0. [1]
Sculley et al. (2015), Hidden Technical Debt in Machine Learning Systems [2]
ISO (2022), ISO/IEC 23053:2022: Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML) [3]

Prof. Dr. Niklas Kühl

AI System Architectures & Interfaces
A predictive API separates model output from authority.

The service exposes capability;
A separate policy or person determines permission to act. [1]

POST /v1/package-defects:predict

REQUEST [2]

```
{
  "image_id": "pkg-1842",
  "line_id": "L2"
}
```

MODEL OUTPUT [2][3]

```
{
  "prediction": "seal_defect",
  "score": 0.92,
  "model_version": "cv-3.4",
  "trace_id": "7A2"
}
```

DECISION POLICY or
HUMAN REVIEW

Assign authority [1]

Parasuraman et al. (2000), A Model for Types and Levels of Human Interaction with Automation [1]
OpenAPI (2025), OpenAPI Specification 3.2.0. [2]
ISO (2022), ISO/IEC 23053:2022: Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML) [3]

Prof. Dr. Niklas Kühl

AI System Architectures & Interfaces
Example of an API.

Example: [1][2]
- ChatGPT
- Python

REQUEST

```python
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-5.6",
    input="Explain the idea behind APIs.",
)
```

MODEL OUTPUT

```
{
  "id": "resp_abcde12345",
  "object": "response",
  "created_at": 1756315696,
  "model": "gpt-5.6",
  "output": [
    {
      "id": "msg_abcde12345",
      "type": "message",
      "status": "completed",
      "content": [
        {
          "type": "output_text",
          "text": "An API (Application Programming Interface) acts as a digital
                   messenger that allows different software applications to securely
                   communicate, share data, and request services from one another."
        }
      ],
      "role": "assistant"
    }
  ]
}
```

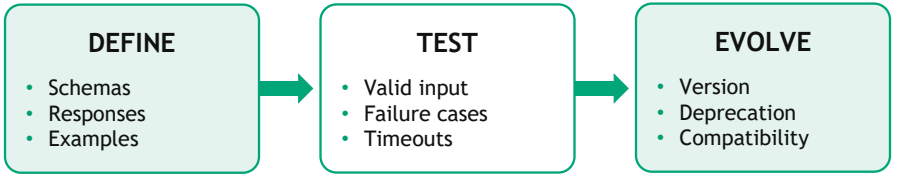
OpenAI (2026), Developer quickstart [1]
OpenAI (2026), Migrate to the Responses API [2]

Prof. Dr. Niklas Kühl

AI System Architectures & Interfaces
Documentation makes an interface testable and evolvable

A machine-readable description lets consumers understand a service without its source code. [1]

```
openapi:3.2.0
paths:
  /v1/package-defects:predict:
    post:
      requestBody: PackageImage
      responses:
        “200”: Prediction
        “422”: InvalidInput
        “503”: Unavailable
```



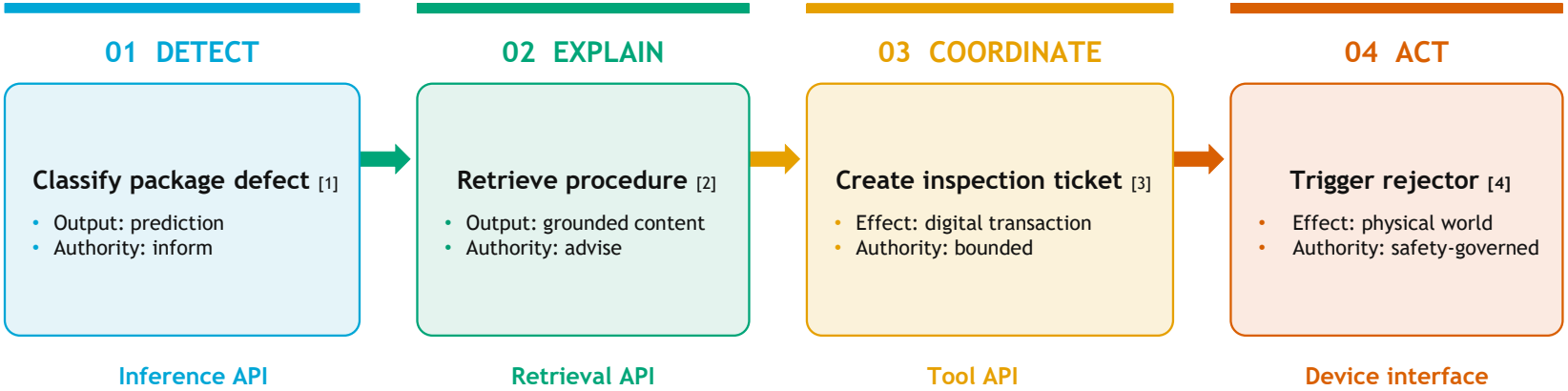
Clear boundaries and documentation reduce fragile code and hidden dependencies. [2]

OpenAPI (2025), OpenAPI Specification 3.2.0. [1]
Sculley et al. (2015), Hidden Technical Debt in Machine Learning Systems [2]

AI System Architectures & Interfaces
APIs make heterogeneous AI components composable.

Schwarz/Kaufland example: [1]

The case progresses from information to content, digital action and physical action.



The architecture combines capabilities;
Authority still remains explicit at every boundary. [5]

Xiong et al. (2023), Designing a computer-vision-based artifact for automated quality control: A case study in the food industry [1]
Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [2]
Schick et al. (2023), Toolformer: Language Models Can Teach Themselves to Use Tools [3]
ISO (2025), ISO 10218-1: Robotics—Safety requirements for industrial robots [4]
Parasuraman et al. (2000), A Model for Types and Levels of Human Interaction with Automation [5]

Prof. Dr. Niklas Kühl

AI System Architectures & Interfaces
Architecture choices become runtime requirements.

Each component brings a different operational constraint into deployment.

PREDICTIVE [1]
- Throughput & latency
- Model versions
- Data drift

RAG [2]
- Knowledge freshness
- Access control
- Retrieval quality

TOOLS [3]
- Credentials
- Transactional effects
- Audit trail

EDGE [4]
- Hard latency
- Local availability
- Safe fallback

Where should each component run?

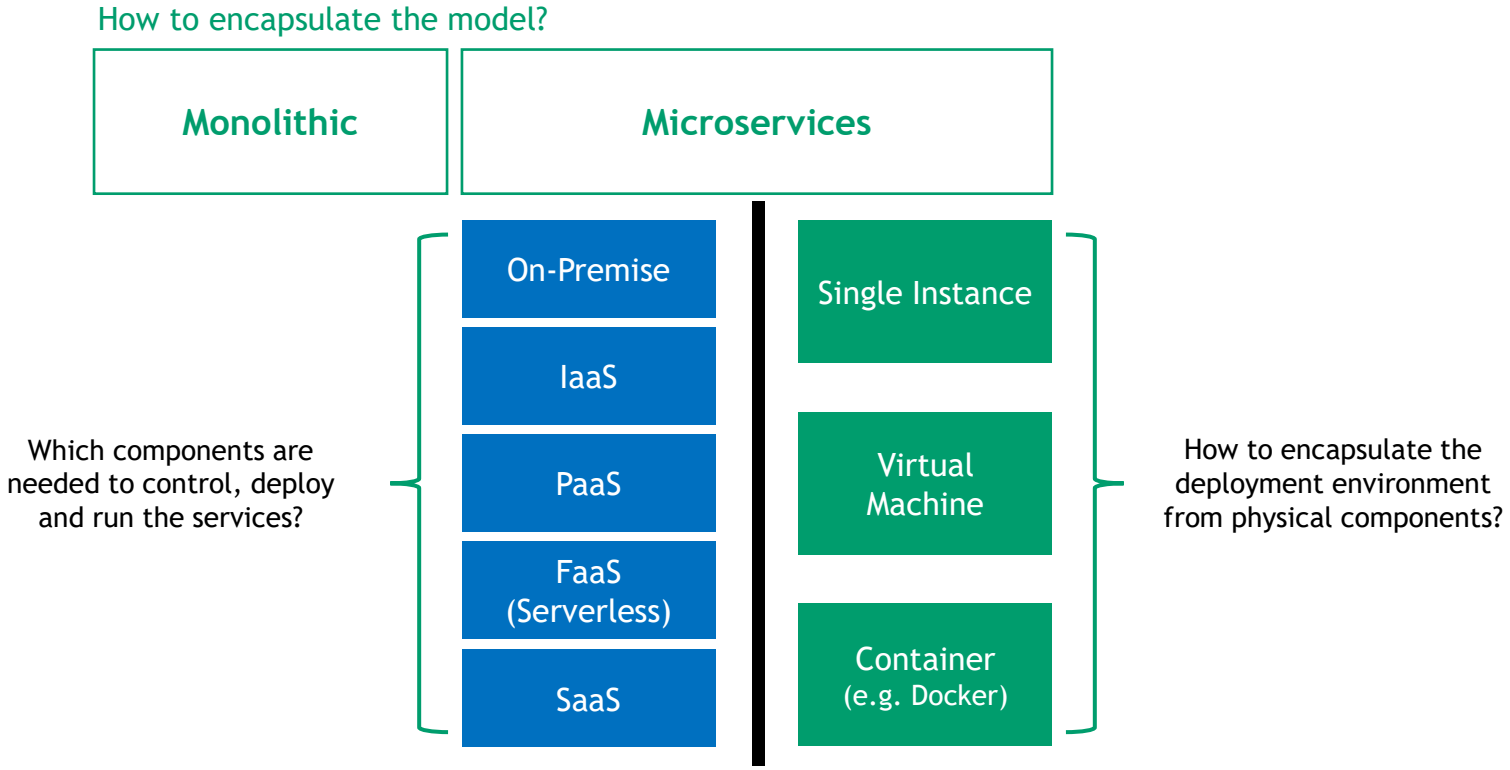
Baylor et al. (2017), TFX: A TensorFlow-Based Production-Scale Machine Learning Platform [1]
NIST (2024), Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile [2]
Allmendinger et al. (2026), Multi-agent AI [3]
Shi et al. (2016), Edge Computing: Vision and Challenges [4]

Prof. Dr. Niklas Kühl



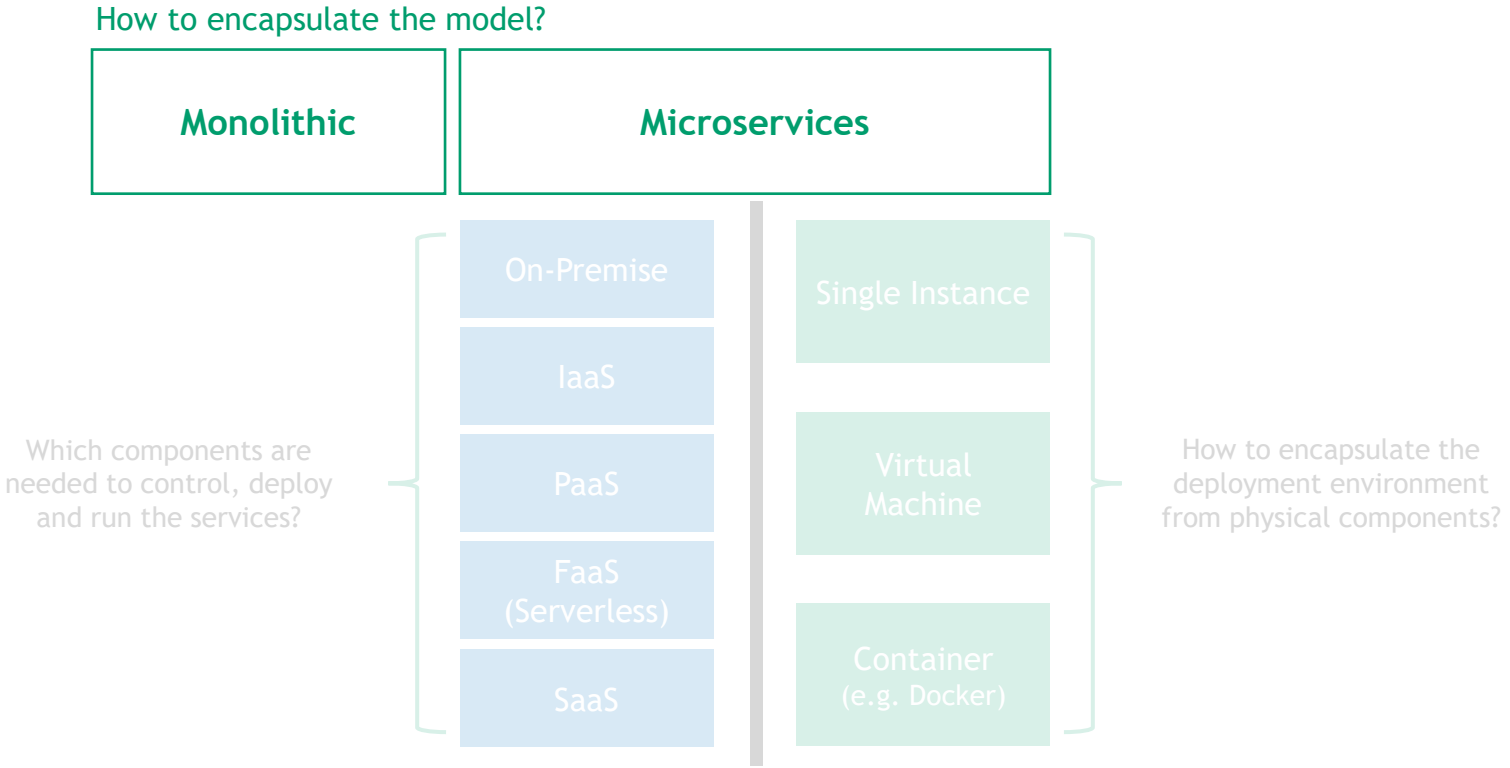
1. From Model to Deployment
2. Levels of Autonomy in AI Deployment
3. AI System Architectures & Interfaces
4. Infrastructure & Runtime Options
5. Monitoring & Runtime Governance
6. MLOps: Continuous Integration, Delivery & Training

Infrastructure & Runtime Options
What are the model deployment decisions?



Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
What are the model deployment decisions?



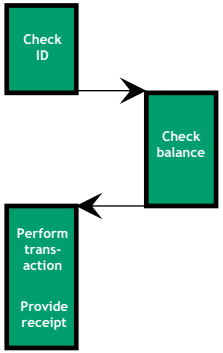
Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
Service-oriented and model-driven programming.



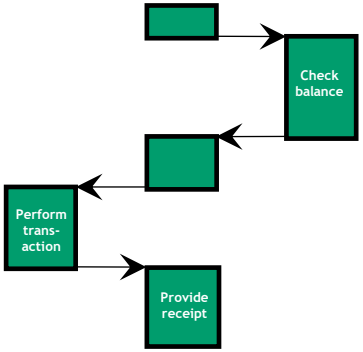
“Monolithic“ Program

One component



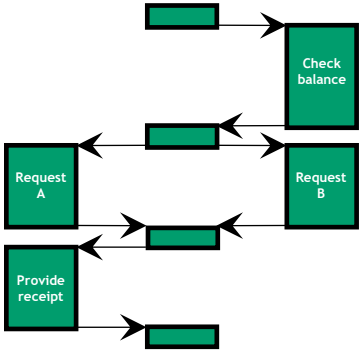
Programming using reusable, pre-programmed components (services)

Two components



Less programming, more use of components (services)

„Many“ components



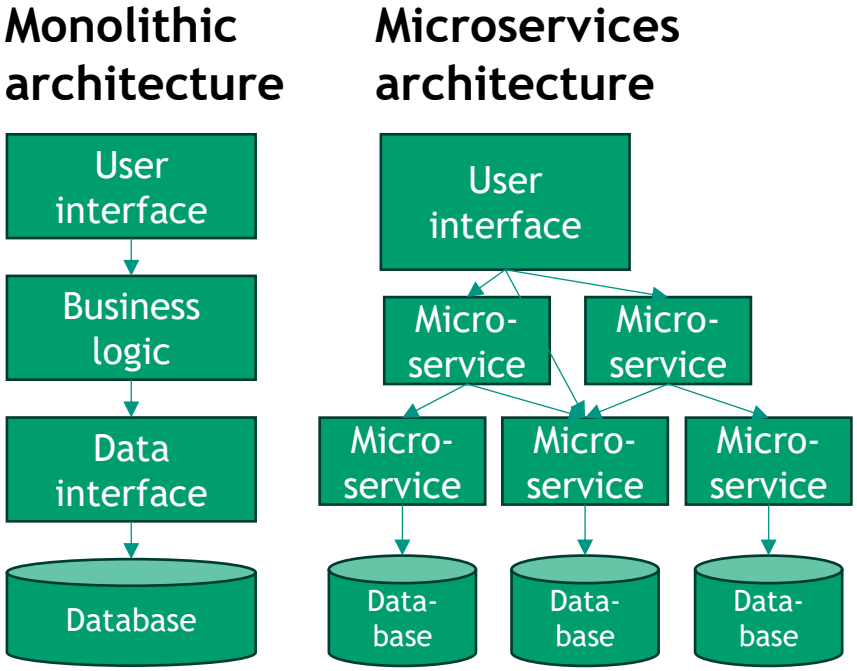
Extensive use of components (services), programming has become “modeling“ or “choreographing“

Simplified example: Banking transaction

Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
What are Microservices?

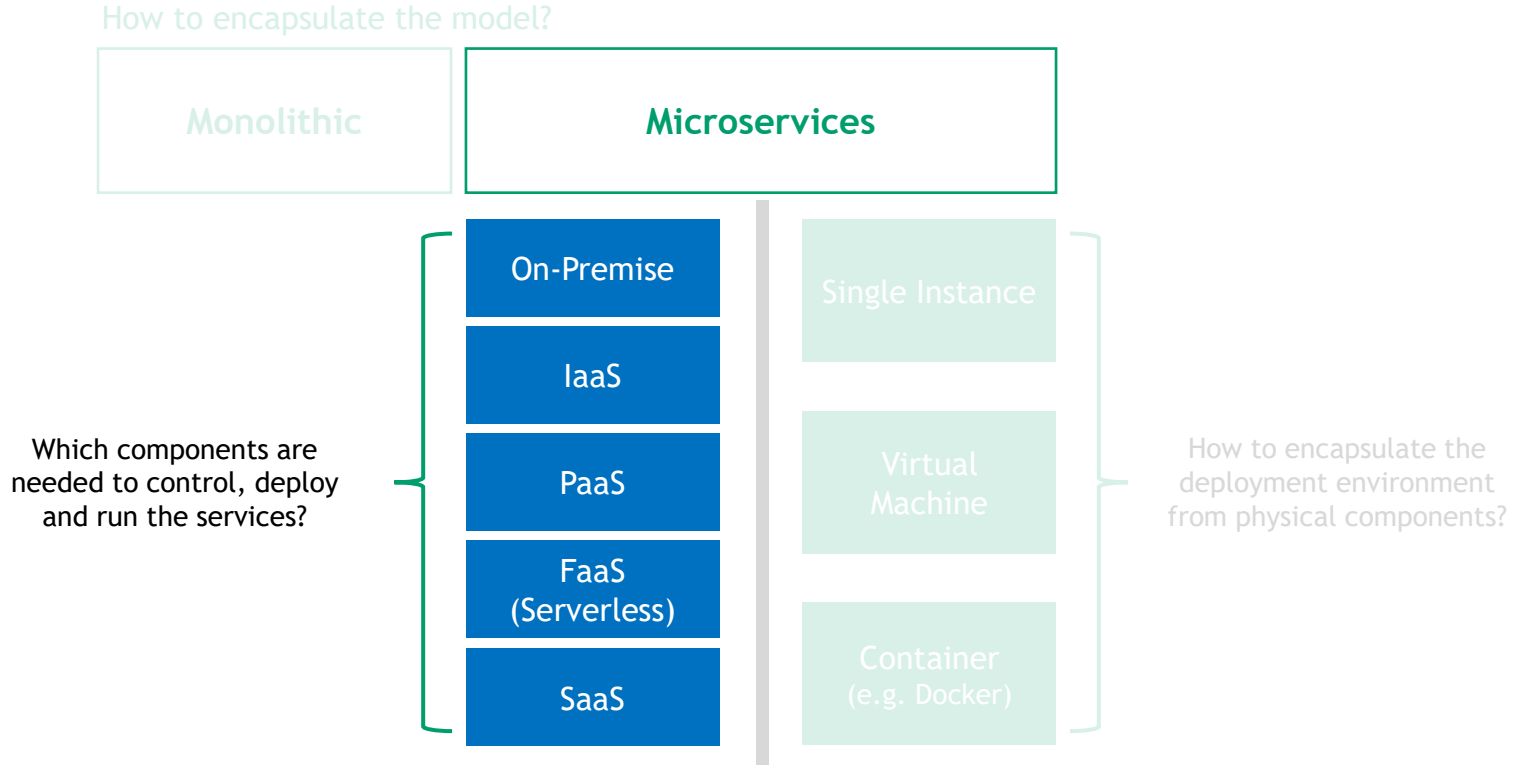
- Microservices – are the next step in the evolution of Service Oriented Architectures. They serve only one specific business function and are completely independent of each other. Microser2vices use lightweight HTTP, REST, or Thrift APIs for communicating among themselves. [1]
- "Almost three-quarters (74%) of respondent organizations are currently using microservices architecture." — Gartner (2023) [2]
- "Microservices are a natural paradigm for developing a machine learning pipeline." — Nitin Bandugula (MapR)



Despodovski (2017), Microservices vs. SOA [1]
Gartner (2023), Microservices Architecture: Have Engineering Organizations Found Success? [2]

Prof. Dr. Niklas Kühl

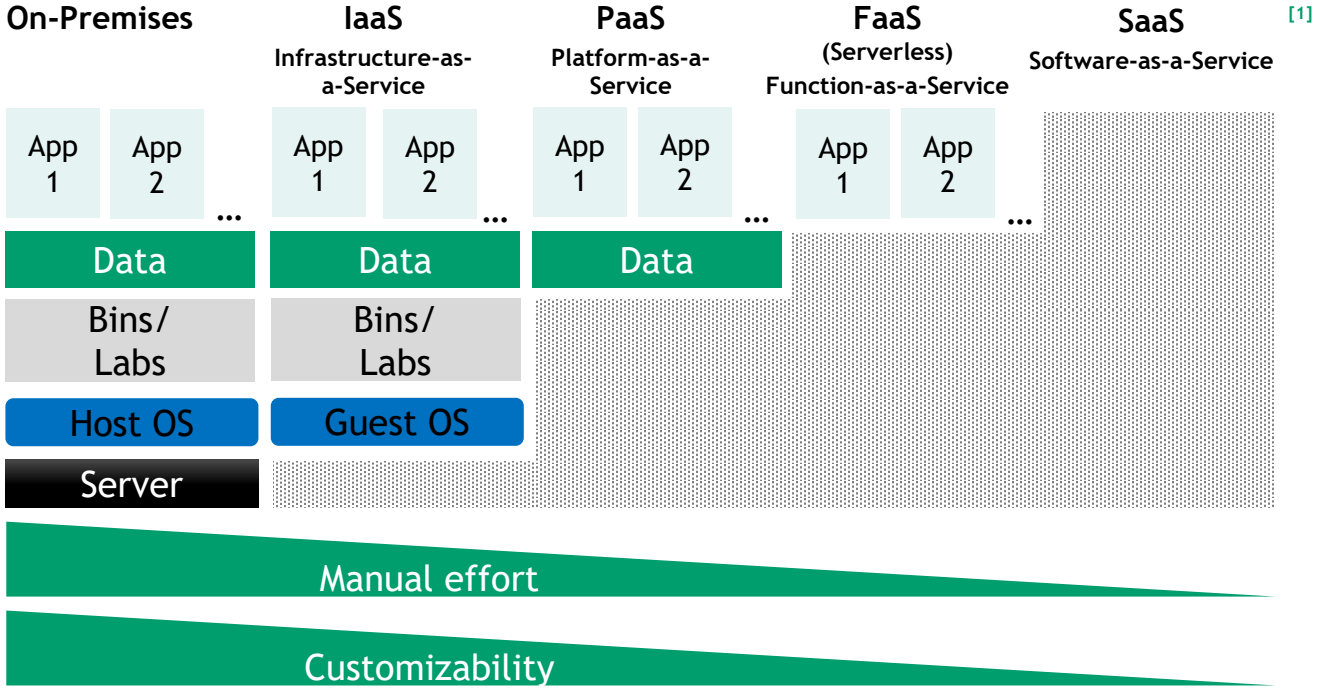
Infrastructure & Runtime Options
Which components are needed?



Prof. Dr. Niklas Kühl

Infrastructure and Runtime Options

Options for (micro)services are manifold.



Peipman (2017), A Short Introduction to Serverless Architecture; Tozzi (2017), Why PaaS vs. CaaS is the wrong Question to ask [1]

Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
Infrastructure-as-a-Service (IaaS).

- Provision, processing, storage, networks, and other fundamental computing resources [1]
- Consumer is able to deploy and run arbitrary software, including operating systems and applications
- Consumer has control over operating systems, storage, and deployed applications; and possibly limited control of select networking components
- Consumer does not manage or control the underlying cloud infrastructure



[2]



[3]


[4]

NIST (2011), The NIST Definition of Cloud Computing [1]
https://commons.wikimedia.org/wiki/File:Amazonwebservices-original-wordmark.svg [2]
https://icon-icons.com/de/symbol/ibm/130909 [3]
https://commons.wikimedia.org/wiki/File:New-azure-logo-square.png [4]

Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
Platform-as-a-Service (PaaS) – Cloud Platforms.

- Capability provided to the consumer to deploy onto the cloud infrastructure consumer-created or acquired applications created using programming languages, libraries, services, and tools supported by the provider
- User has control over the deployed applications and possibly configuration settings for the application-hosting environment
- Consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, or storage



[1]


Google App Engine
[2]

https://commons.wikimedia.org/wiki/File:Salesforce_Cloud_Alliance.jpg [1]
https://icon-icons.com/de/symbol/google/30031 [2]

Prof. Dr. Niklas Kühl

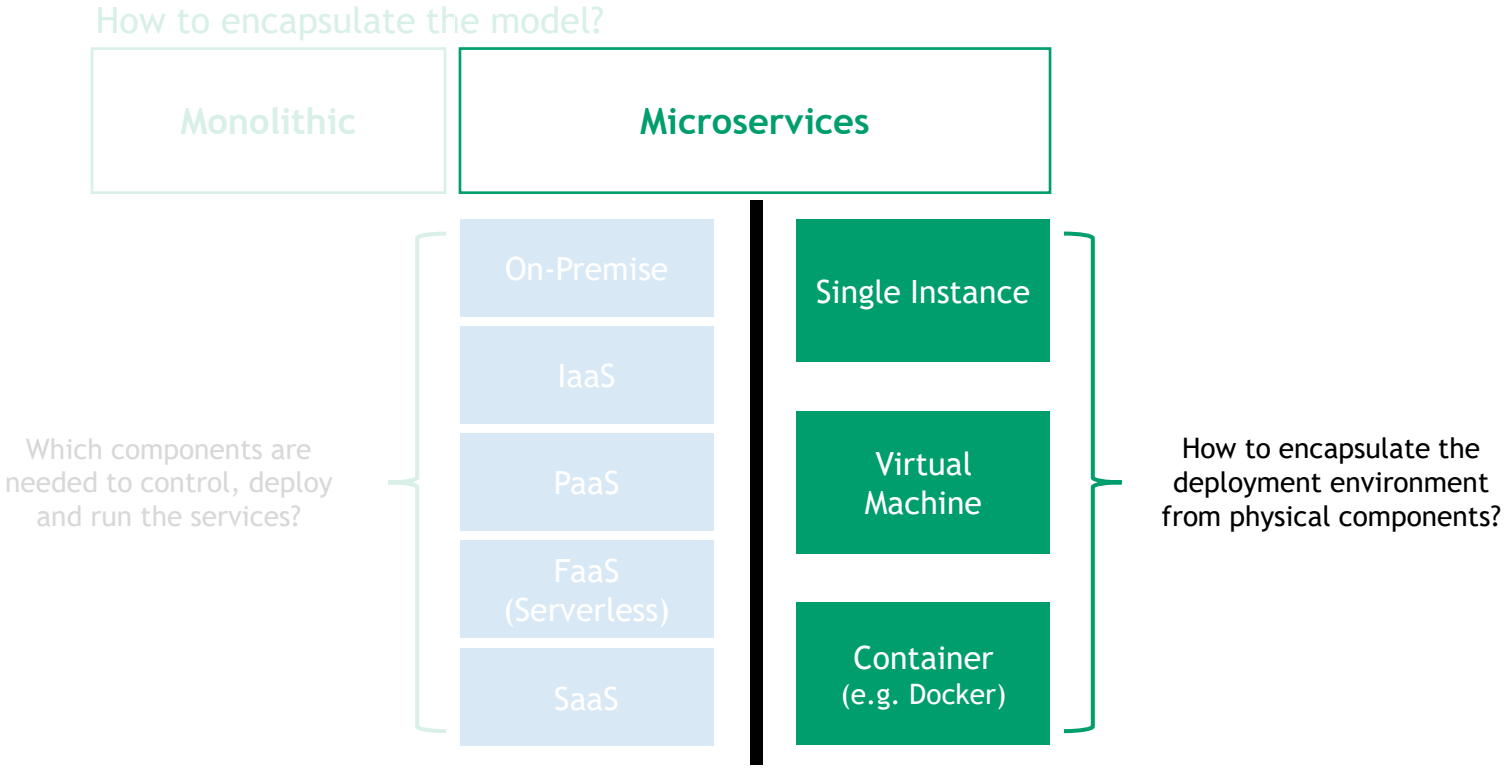
Infrastructure & Runtime Options
FaaS (Serverless) – Advantages.

- Function-as-a-Service – is a model to run pure programming code. The service provider deals with every physical or software component besides the actual function.
  - Pay – only when the function is running. No costs for unused server.
  - Server management and security – is provided by the service provider.
  - Deployment – is simple, only the code snippet is needed.
  - Scale – the function seamlessly and almost unlimited.

Further reading: L Schuler, S Jamil, N Kühl: AI-based Resource Allocation: Reinforcement Learning for Adaptive Auto-scaling in Serverless Environments

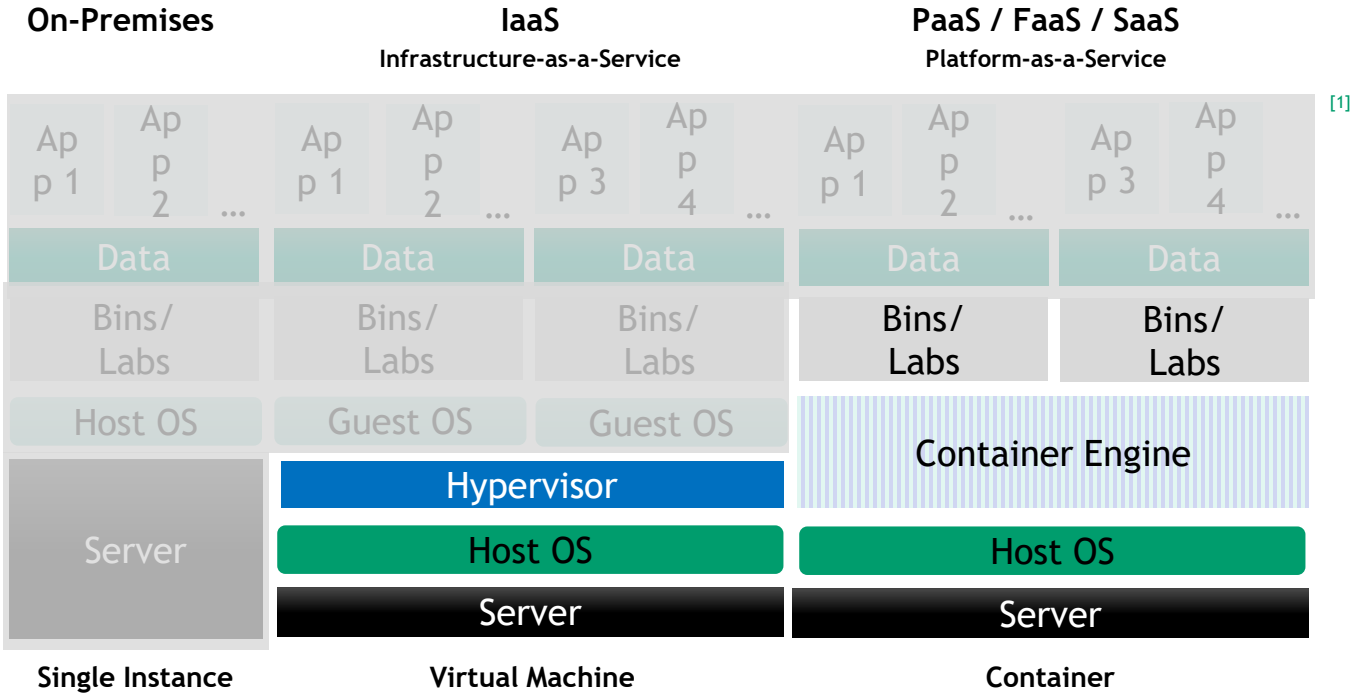
Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
What are the model deployment decisions?



Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
Virtual Machines & Containers to encapsulate deployment.

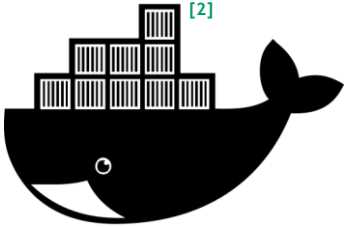


Peipman (2017), A Short Introduction to Serverless Architecture; Tozzi (2017), Why PaaS vs. CaaS is the wrong Question to ask [1]

Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
Advantages of Container.

- Container – enables an abstraction of applications from the environment in which they run. A container contains a single application including its dependencies but without the full OS to run on. [1]
  - Portability – of containers are given because container contains every dependency and configuration.
  - Efficiency – by not using an entire OS, emulated by a virtual machine
  - Isolation – provides a consistent environment. Every application runs in its own sandbox
  - Scalability – enables to assign more resources to one container but also adding more containers to the same container engine
- **Docker** – is the best-known container provider. Its standardized container file format further helps portability.

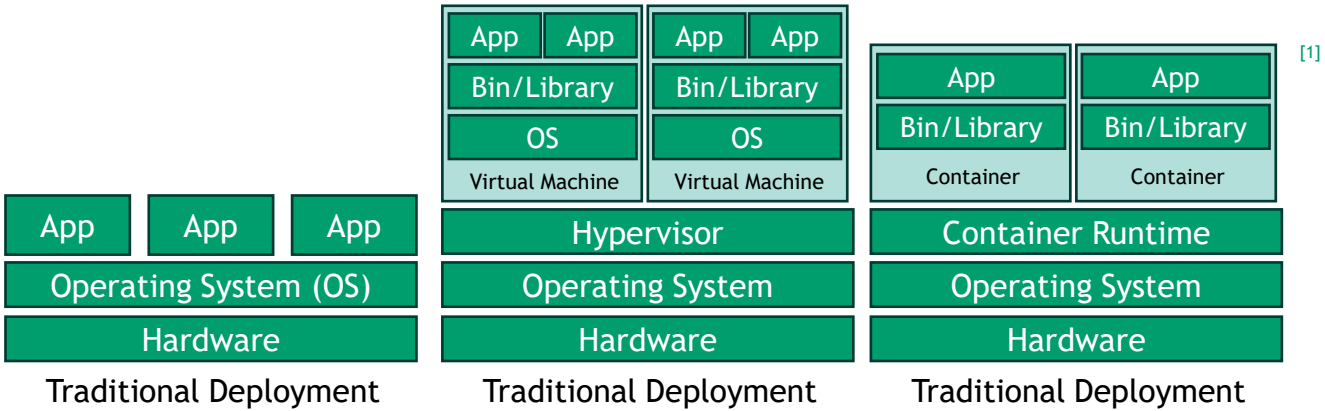

[2]

Peipman (2017), A Short Introduction to Serverless Architecture; Tozzi (2017), Why PaaS vs. CaaS is the wrong Question to ask [1]
https://icon-icons.com/icon/Docker/132435 [2]

Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
Exemplary deployment with Kubernetes.

- Kubernetes – is a system for managing containers. It enables deploying, maintenance and scaling of containerized applications often across multiple host servers.
  - Pods – bundle several containers on the same host server.
  - Nodes – bundle several pods. All nodes form a cluster.
  - Master Plane – controls the cluster. It communicates with the cluster and schedules tasks.



[1]

Bernstein (2014), From lxc to docker to Kubernetes, Kubernetes.io, What is Kubernetes? [1]

Prof. Dr. Niklas Kühl

Infrastructure & Runtime Options
Exemplary deployment with Cloud Foundry.

- Cloud Foundry – is a platform-as-a-service to develop and scale applications. [1]
  - External resources – such as databases, storage or communication are provided by Cloud foundry as services. Applications have to use these services to e.g. store their internal state. External provides can register their API in the cloud foundry marketplace to be used by applications deployed by Cloud Foundry.
  - Support for programming languages and frameworks – is built in for many common use cases.
  - Scalability – ensued automatically for resources and CPU requirement
  - Logging and monitoring – is provided by Cloud Foundry.



CLOUD **FOUNDRY** [2]

Bernstein (2014), Cloud foundry aims to become the OpenStack of PaaS [1]
https://icon-icons.com/icon/file-type-light-cloudfoundry/130474 [2]

Prof. Dr. Niklas Kühl

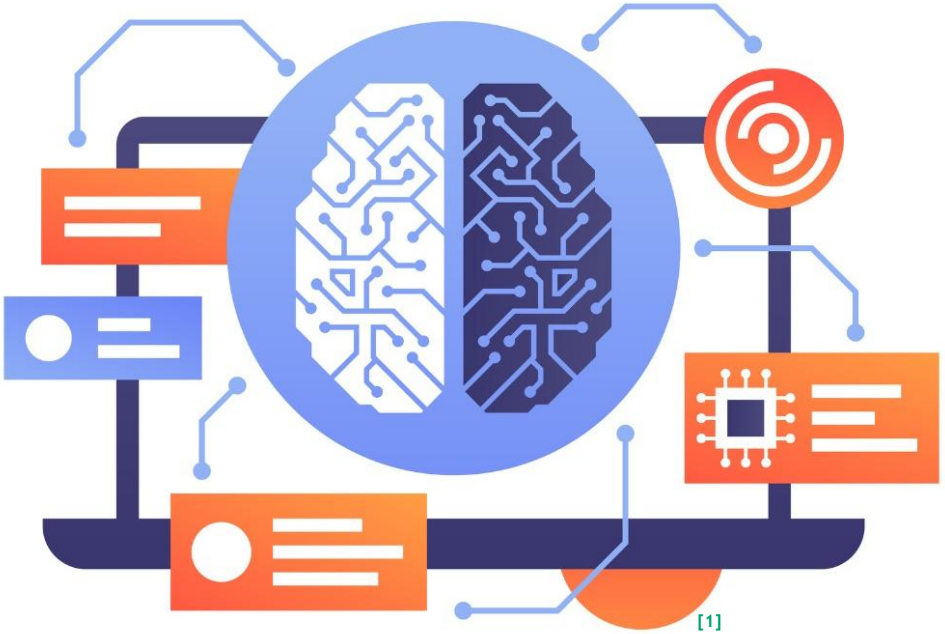
Infrastructure & Runtime Options

Deployment with managed services (MLaaS = SaaS).

Azure Machine Learning services

Amazon SageMaker

IBM Watson Studio



[1]

https://www.deanlong.io/blog/the-google-search-machine-learning-we-trust [1]

Prof. Dr. Niklas Kühl



1. From Model to Deployment
2. Levels of Autonomy in AI Deployment
3. AI System Architectures & Interfaces
4. Infrastructure & Runtime Options
5. Monitoring & Runtime Governance
6. MLOps: Continuous Integration, Delivery & Training

Monitoring & Runtime Governance

Runtime governance is a closed loop from evidence to intervention.

Control starts with evidence and ends with accountable improvement. [1][2]



The **loop** spans the **complete deployed system**—not only the model. [1]

NIST (2023), Artificial Intelligence Risk Management Framework [1]
ISO (2023), ISO/IEC 42001:2023: Information technology—Artificial intelligence—Management system [2]

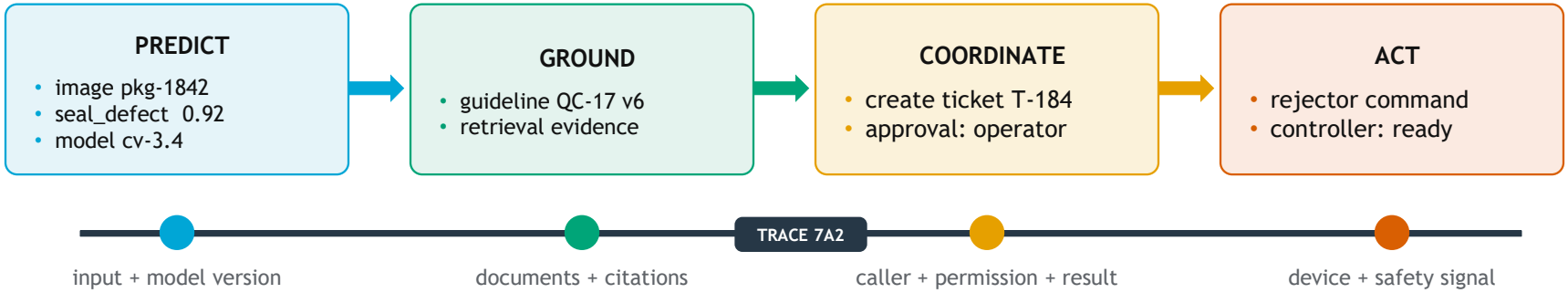
Prof. Dr. Niklas Kühl

Monitoring & Runtime Governance

Observability reconstructs what the system saw, decided and did.

A shared trace connects model behavior, knowledge, tool effects and machine state. [1][2][3]

Example:



Latency and **error** rates show **service health**;
The **trace** explains **system behavior.** [1][3]

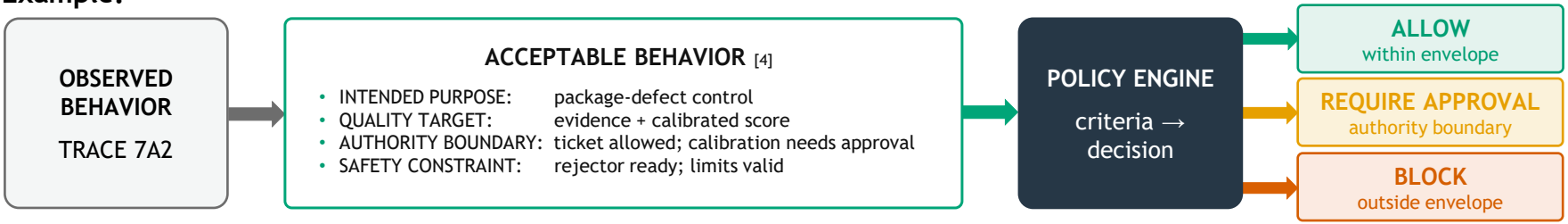
Breck et al. (2017), The ML Test Score: A rubric for ML Production Readiness and Technical Debt Reduction [1]
W3C (2021), Trace Context [2]
EU AI Act (2024), Regulation (EU) 2024/1689, Article 12: Record-keeping [3]

Prof. Dr. Niklas Kühl

Monitoring & Runtime Governance
Alignment compares observed behavior with explicit criteria.

Observed behavior becomes governable only after the acceptable envelope is defined. [1][2][3]

Example:



Policies operationalize alignment;
They do **not resolve** whose **values** should define the **criteria.** [1][3]

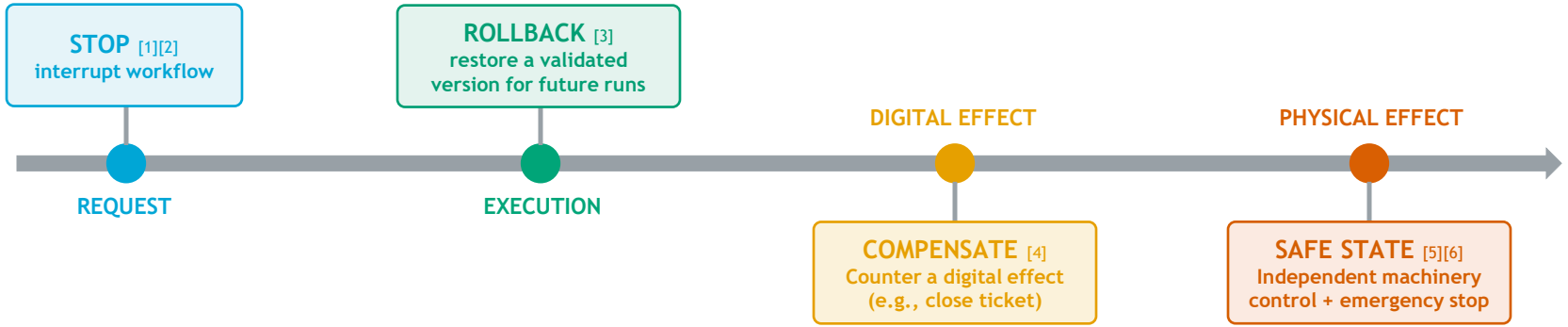
NIST (2023), Artificial Intelligence Risk Management Framework [1]
ISO (2023), ISO/IEC 23894:2023: Information technology—Artificial intelligence—Guidance on risk management [2]
Gabriel (2020), Artificial Intelligence, Values, and Alignment [3]
Allmendinger et al. (2026), Multi-agent AI [4]

Prof. Dr. Niklas Kühl

Monitoring & Runtime Governance

Interventions must match the reversibility of their effects.

Rollback changes future executions;
It does not undo an action that already occurred. [1][4]



Physical safeguards remain **independent** of **AI policy** and **model availability.** [5][6]

EU AI Act (2024), Regulation (EU) 2024/1689, Article 14: Human Oversight [1]
NIST (2023), Artificial Intelligence Risk Management Framework [2]
Google Cloud Architecture Center (2024), MLOps: Continuous Delivery and Automation Pipelines in Machine Learning [3]
Chang et al. (2025), SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning [4]
ISO (2023), ISO 13849-1:2023: Safety of machinery—Safety-related parts of control systems—Part 1: General principles for design [5]
ISO (2015): ISO 13850:2015: Safety of machinery—Emergency stop function—Principles for design [6]

Prof. Dr. Niklas Kühl

Monitoring & Runtime Governance
Oversight requires assigned authority before incidents occur.

Human oversight is a designed capability to monitor, intervene and escalate—not approval for every action. [1][2]

Example:

LINE OPERATOR	Escalation	Stop local process → confirm safe state → escalate
AI SERVICE OWNER		Disable endpoint → diagnose → initiate rollback
QUALITY & SAFETY OWNER		Approve restart → approve policy-limit change
MLOPS TEAM [3]		Implement change → add evaluation case → validate → release



EU AI Act (2024), Regulation (EU) 2024/1689, Article 14: Human Oversight [1]
NIST (2023), Artificial Intelligence Risk Management Framework [2]
Kreuzberger et al. (2023), Machine Learning Operations (MLOps): Overview, Definition and Architecture [3]

Prof. Dr. Niklas Kühl



1. From Model to Deployment
2. Levels of Autonomy in AI Deployment
3. AI System Architectures & Interfaces
4. Infrastructure & Runtime Options
5. Monitoring & Runtime Governance
6. MLOps: Continuous Integration, Delivery & Training

MLOps: Continuous Integration, Delivery & Training
Why is Machine Learning Operations (MLOps) interesting?

"What Data Scientists are taught"

```
In [24]: # from sklearn import DecisionTreeClassifier
         from sklearn.tree import DecisionTreeClassifier
         from sklearn.metrics import accuracy_score

         my_random_seed = 42

         # Initialize the DecisionTreeClassifier
         model = DecisionTreeClassifier(random_state=my_random_seed)

         # train the classifier
         model.fit(X_train, y_train)

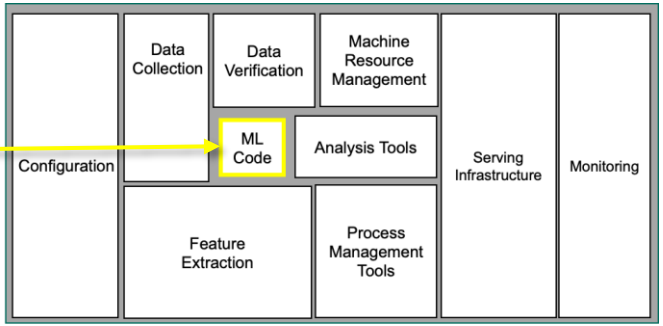
         # Predict with the test data
         y_predict = model.predict(X_test)

         # Determine the accuracy score
         accuracy_score = accuracy_score(y_test, y_predict)

         print ("Accuracy score for the trained model: {:.4f}.".format(accuracy_score))

         Accuracy score for the trained model: 1.0000.
```

"What reality looks like "



[2]

Source: Hidden Technical Debt in Machine Learning Systems | (D. Sculley et. al., 2015)

Solution: MLOps

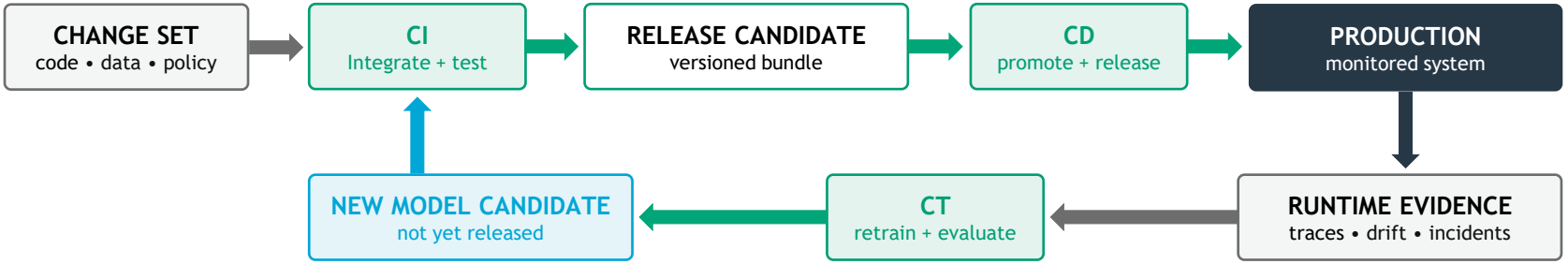
"MLOps (Machine Learning Operations) is a paradigm, including aspects like best practices, sets of concepts, as well as a development culture when it comes to the end-to-end conceptualization, implementation, monitoring, deployment, and scalability of machine learning products.
Most of all, it is an engineering practice that leverages three contributing disciplines: machine learning, software engineering (especially DevOps), and data engineering. MLOps is aimed at productionizing machine learning systems by bridging the gap between development (Dev) and operations (Ops). Essentially, MLOps aims to facilitate the creation of machine learning products by leveraging these principles: CI/CD automation, workflow orchestration, reproducibility; versioning of data, model, and code; collaboration; continuous ML training and evaluation; ML metadata tracking and logging; continuous monitoring; and feedback loops." [1]

Kreuzberger et al. (2023), Machine Learning Operations (MLOps): Overview, Definition, and Architecture [1]
Iliadis et al. (2022), The Seer and the Seen: Surveying Palantir's Surveillance Platform [2]

Prof. Dr. Niklas Kühl

MLOps: Continuous Integration, Delivery & Training
MLOps connects runtime evidence to controlled releases.

CI (Continuous Integration) **validates changes, CD** (Continuous Delivery) **promotes releases,**
CT (Continuous Training) **creates new model candidates.** [1][2][3]



The **three practices** form one **controlled lifecycle**—not a fixed linear sequence.

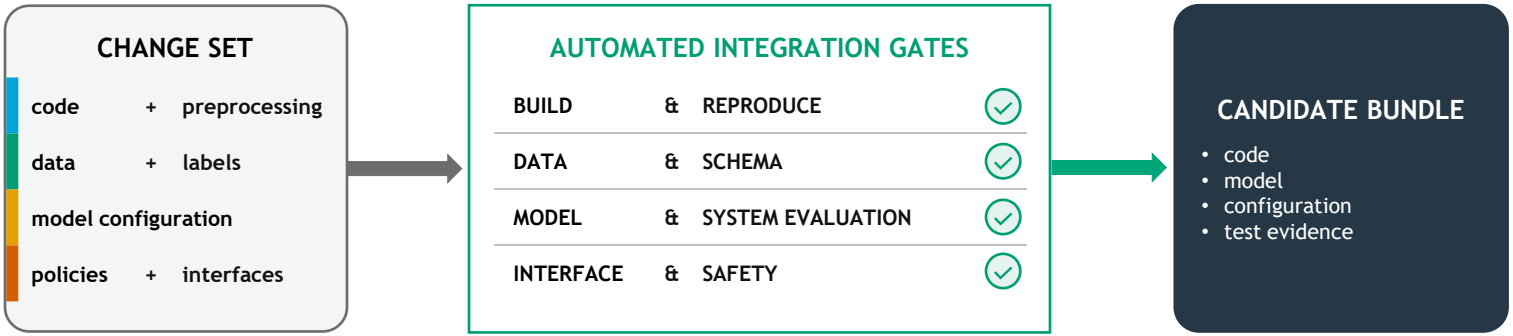
Kreuzberger et al. (2023), Machine Learning Operations (MLOps): Overview, Definition and Architecture [1]
Steidl et al. (2023), The pipeline for the continuous development of artificial intelligence models—Current state of research and practice [2]
ISO (2023), ISO/IEC 5338:2023: Information technology—Artificial intelligence—AI system life cycle processes [3]

Prof. Dr. Niklas Kühl

MLOps: Continuous Integration, Delivery & Training
CI makes every AI change testable before release.

A commit is accepted only when predefined engineering and ML-specific conditions pass. [1][2][3]

Example:



Pass/fail criteria are defined **before** the candidate is **evaluated.** [1][2]

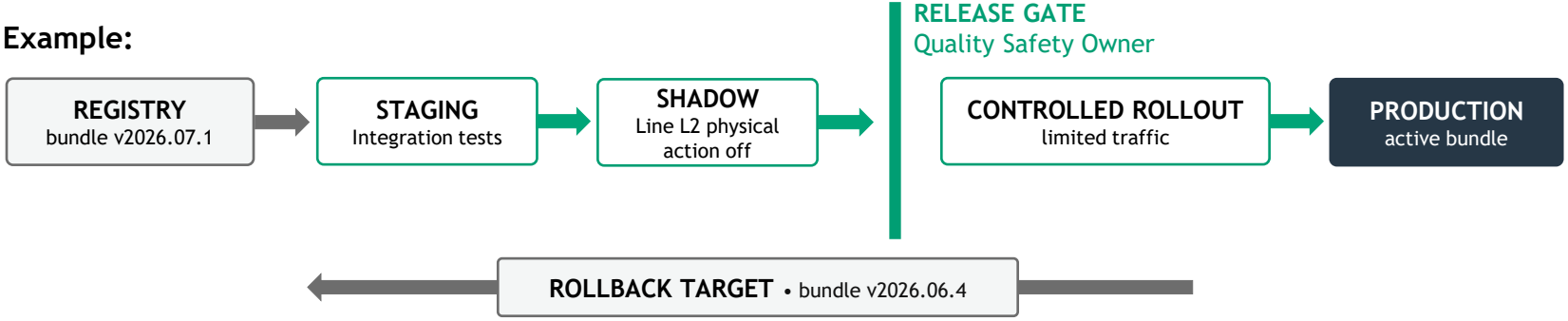
Renggli et al. (2019), Continuous Integration of Machine Learning Models with ease.ml/ci: Towards a Rigorous Yet Practical Treatment [1]
Breck et al. (2017), The ML Test Score: A rubric for ML Production Readiness and Technical Debt Reduction [2]
Sculley et al. (2015), Hidden Technical Debt in Machine Learning Systems [3]

Prof. Dr. Niklas Kühl

MLOps: Continuous Integration, Delivery & Training
CD promotes versioned artifacts through controlled environments.

Delivery keeps a validated release deployable;
Production activation may remain gated. [1][2]

Example:



CD is **not** the same as **automatic deployment.** [1][3]

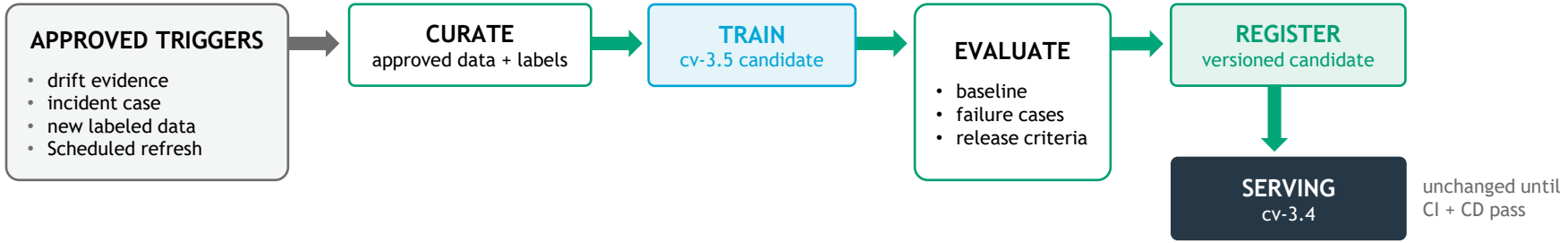
Shahin et al. (2017), Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices [1]
ISO (2023), ISO/IEC 5338:2023: Information technology—Artificial intelligence—AI system life cycle processes [2]
Kreuzberger et al. (2023), Machine Learning Operations (MLOps): Overview, Definition and Architecture [3]

Prof. Dr. Niklas Kühl

MLOps: Continuous Integration, Delivery & Training
CT creates a new candidate—not an unreviewed live model.

Continuous means repeatable and triggerable—not always-on adaptation. [1][2][3]

Example:



The serving **model** remains **unchanged** until **CI** and **CD accept** the candidate.

Baylor et al. (2017), TFX: A TensorFlow-Based Production-Scale Machine Learning Platform [1]
Kreuzberger et al. (2023), Machine Learning Operations (MLOps): Overview, Definition and Architecture [2]
Steidl et al. (2023), The pipeline for the continuous development of artificial intelligence models—Current state of research and practice [3]

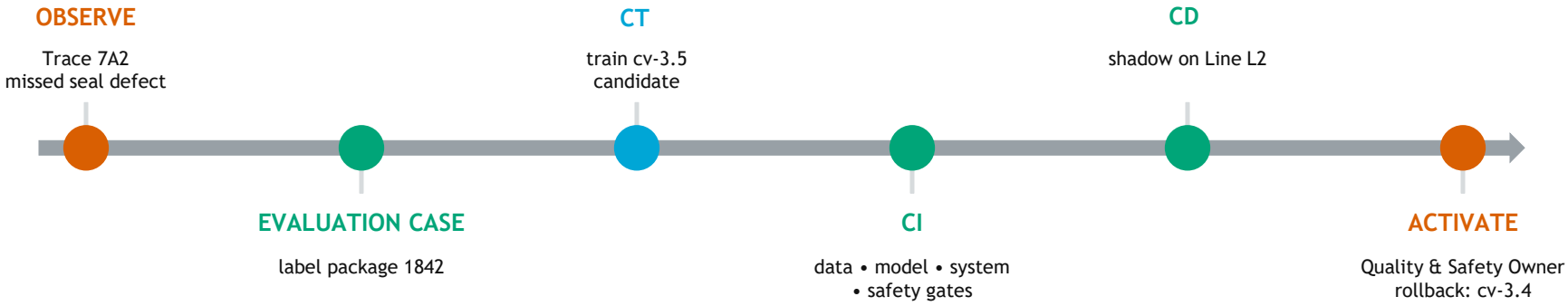
Prof. Dr. Niklas Kühl

MLOps: Continuous Integration, Delivery & Training

One defect event can trigger CI, CT and CD without bypassing governance.

A monitored failure becomes a versioned change with an accountable release decision. [1][2]

Example:



Every change remains **traceable** from incident to release—and back to the previous version.

Kreuzberger et al. (2023), Machine Learning Operations (MLOps): Overview, Definition and Architecture [1]
NIST (2023), Artificial Intelligence Risk Management Framework [2]

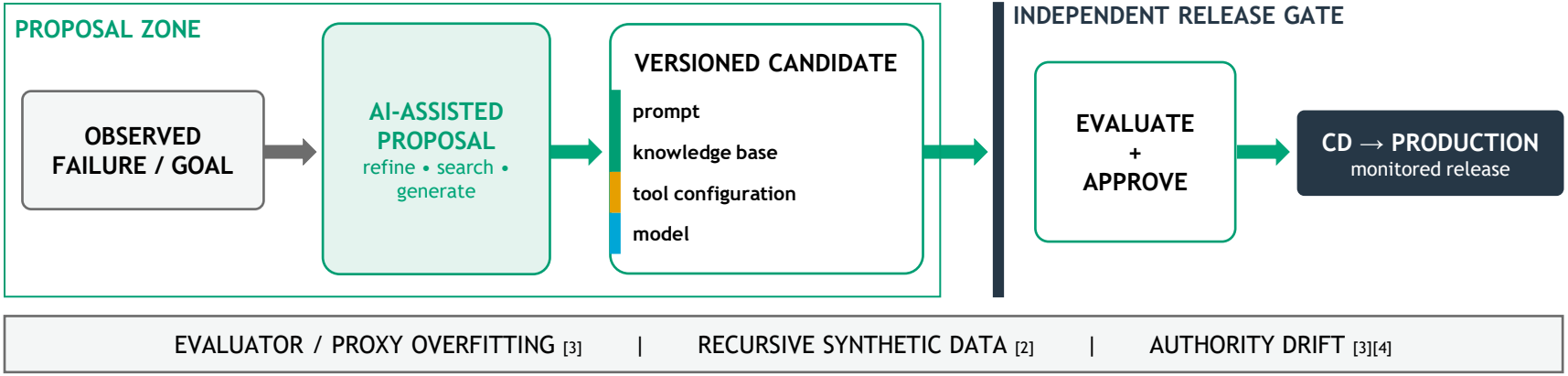
Prof. Dr. Niklas Kühl

MLOps: Continuous Integration, Delivery & Training

Self-improving AI should produce candidates—not silently rewrite production.

Self-refinement can improve outputs, but recursive feedback also creates new failure modes. [1][2]

Example:



The **more** of the loop **AI** closes, the **more independent evaluation** and **release authority** must become. [3][4]

Madaan et al. (2023), Self-Refine: Iterative Refinement with Self-Feedback [1]
Shumailov et al. (2024), AI models collapse when trained on recursively generated data [2]
NIST (2023), Artificial Intelligence Risk Management Framework [3]
ISO (2023), ISO/IEC 42001:2023: Information technology—Artificial intelligence—Management system [4]

Prof. Dr. Niklas Kühl

Summary

Finalize the Candidate

Deployment begins with a locked configuration, approved data and a versioned model bundle ready for system integration.

Assign Authority

Predictive, generative, agentic and physical AI differ in what they produce—and in how much authority they should receive.

Compose the System

Models become deployable through data, rules and interfaces: inference, retrieval, tool and device APIs compose the operational system.

Place the Runtime

Latency, availability, data locality and safety determine whether components run on-premises, in the cloud or at the edge.

Govern Operation

Observability reconstructs what the system saw, decided and did; policies, oversight, stop and recovery keep behavior within bounds.

Improve Safely

Runtime evidence creates new candidates. CI validates changes, CT trains candidates and CD promotes approved releases—never silent production updates.

Prof. Dr. Niklas Kühl